

★ Lesson 1 — Understanding the Problem, Dataset & Fairness Goal

Foundation for a Winning ISEF / AI Competition Project

1. Lesson Summary

This lesson introduces the core scientific problem your project is trying to solve: ➡ **Demographic bias in intent recognition models** ➡ **Why current models fail**, especially for minority speakers ➡ **What trustworthy intent means** ➡ **What the dataset looks like (1,152 utterances, 96 speakers)** ➡ **Why adversarial debiasing is needed for fairness**

By the end of Lesson 1, the student will deeply understand the domain background, the data they will use, the scientific hypothesis, and the fairness gaps that must be solved—exactly what top ISEF judges expect.

2. Key Points

- Trustworthiness perception is strongly influenced by **voice acoustics**.
 - Most research suffers from **White Western Individualist Bias (WWIB)**.
 - The dataset includes **1,152 utterances from 96 speakers** across **White, Black, and South Asian** groups.
 - The baseline Random Forest model only achieved **~71% accuracy**, with large gaps across ethnic groups.
 - The project's scientific goal: **reduce demographic performance gaps** while increasing accuracy.
 - Deep learning on Mel-Spectrograms allows learning **universal prosodic cues**.
 - Adversarial debiasing forces the model to **ignore speaker identity**.
 - This project has strong real-world value: fairer speech technology + TrustTrainer app.
 - ISEF judges look for **clarity of hypothesis, fairness motivation, and innovation**.
-

3. Real-World Examples or Stories

- **Hiring & interviews:** Voice-based AI screening systems have been shown to score some accents lower in "confidence" or "friendliness," even when content is identical.
- **Voice assistants (Siri/Alexa):** Studies show that these systems struggle more with **Black and South Asian English speakers**, reinforcing inequality.
- **Courtroom / law enforcement tech:** Poor calibration on minority voices can amplify bias.

These help the student explain the *real significance* of the project—ISEF judges value this.

4. Terminology Explained

Term	Simple Explanation
Trustworthy Intent	Whether a speaker sounds sincere, honest, or believable.
Prosody	Rhythm, pitch, tone—how something is said rather than the words themselves.
Mel-Spectrogram	A picture of sound showing frequencies and intensity over time.
Bias (in ML)	When a model performs worse for certain groups.
Demographic Parity	Balanced performance across demographic subgroups.
Adversarial Debiasing	Training a model to perform well while <i>unlearning</i> sensitive attributes (e.g., ethnicity).
Gradient Reversal Layer (GRL)	A trick that tells the network to get worse at predicting sensitive attributes.

5. How It Works — Real-World Application

Step-by-step workflow of this project:

1. **Load raw audio (.wav)**
2. **Convert to Mel-Spectrograms** — easier for CNNs to learn patterns
3. **Build a CNN classifier**
4. **Add adversarial head (Ethnicity/Age classifier)**
5. **Reverse gradients** so feature extractor forgets demographic cues
6. **Train jointly**
 - Main head → maximize trust classification
 - Adversarial head → minimize its accuracy
7. **Evaluate fairness**
 - Compare White vs Black vs South Asian accuracy & recall
8. **Deploy in TrustTrainer app**

This connects the science → AI → fairness → real product pipeline.

6. Practice Exercises (5)

Exercise 1 — Identify Bias Sources

List 5 ways demographic bias can enter a speech dataset.

Exercise 2 — Dataset Breakdown

Compute how many utterances per demographic group if the dataset has:

- 96 speakers
- 3 groups
- 12 samples per speaker

Exercise 3 — Prosody Examples

Write 3 everyday sentences and describe how tone can make them more or less trustworthy.

Exercise 4 — Spectrogram Exploration (No coding yet)

Find any audio file on your computer and use an online tool to visualize its spectrogram. Describe what you notice.

Exercise 5 — Research Reflection

In 150 words, explain why “White Western Individualist Bias” is harmful in AI systems.

7. Solutions

Solution 1

Bias sources:

1. Unbalanced dataset
2. Accent variety differences
3. Microphone quality differences
4. Age distribution differences
5. Cultural differences in prosody

Solution 2

$96 \text{ speakers} \div 3 \text{ groups} = 32 \text{ speakers per group}$
 $32 \times 12 \text{ utterances} = \mathbf{384 \text{ utterances per group}}$

Solution 3

Examples include tone of excitement, slow vs fast speaking, confidence, nervous pauses.

Solution 4

Students should mention:

- Bright areas = strong frequencies
- Dark areas = quiet
- Speech has repeating formant shapes

Solution 5

A strong reflection should touch on fairness, societal harm, and reduced accuracy for minorities.

8. Q&A (10 Common Student Questions)

1. **Why can't we just use pitch/jitter features like the paper?** → Deep learning can discover *hidden patterns* that manual features miss.
 2. **Why do speech models become biased?** → Uneven representation + models overfitting to identity cues.
 3. **What if the dataset is too small?** → Use augmentation: noise, pitch shift, time stretch.
 4. **Why Mel-spectrograms?** → They compress human speech frequencies in a human-hearing-like way.
 5. **Why CNNs?** → Their filters learn patterns in images, which spectrograms basically are.
 6. **What is adversarial training?** → A method to force the model to forget sensitive attributes.
 7. **Can this project win ISEF?** → Yes—fairness + innovation + real app is very competitive.
 8. **Why not use LLMs?** → This is a prosody-based task, not text-based.
 9. **What if the adversarial head performs too well?** → Increase gradient reversal strength so the network unlearns more.
 10. **How do judges evaluate fairness?** → They look at subgroup metrics: recall, precision, F1.
-

9. Quiz (10 Questions)

1. **What is prosody?** A. Voice tone & rhythm ✓ B. Text meaning C. Speaker gender D. Noise level
 2. **What is the main fairness issue in the dataset?** Correct: **Unequal performance across ethnic groups**
 3. **How many total utterances?** Correct: **1,152**
 4. **Baseline Random Forest accuracy?** Correct: **~71%**
 5. **Which representation is used for CNN input?** Correct: **Mel-Spectrograms**
 6. **What does the adversarial head predict?** Correct: **Demographics (Ethnicity/Age)**
 7. **What does the GRL do?** Correct: **Reverses gradients to remove demographic cues**
 8. **What metric compares fairness?** Correct: **Recall gap or accuracy gap**
 9. **Why audio augmentation?** Correct: **Increase robustness & dataset size**
 10. **Why is this project novel?** Correct: **Adversarial debiasing for trust intent recognition**
-

10. Mini Practice Project (with downloadable .zip)

Mini Project: Explore Prosody & Spectrograms

The student will:

1. Load a sample .wav file
 2. Convert it into a Mel-Spectrogram
 3. Visualize it using matplotlib
 4. Save the spectrogram as an image
 5. Write a short interpretation
-

11. References

From the attached project plan

Directly used throughout this lesson.

Additional learning resources

- Librosa tutorial: <https://librosa.org/doc/latest/tutorial.html>
 - Torchaudio tutorial: https://pytorch.org/tutorials/beginner/audio_preprocessing_tutorial.html
 - Fairness in ML (Google): <https://developers.google.com/machine-learning/fairness-overview>
 - Adversarial Debiasing (ARXIV): <https://arxiv.org/abs/1801.07593>
 - Spectrogram basics: <https://www.youtube.com/watch?v=4SHbg-U5cKY>
-

12. Additional Information (Expert Tips for ISEF Competitiveness)

- Judges love **clear hypotheses** → Here: *"Adversarial debiasing will reduce demographic performance gaps."*
 - They also value **domain significance** → Bias in speech tech has real societal impact.
 - Documentation is key — Keep a **research logbook** and record every experiment.
 - Charts matter: Show subgroup metrics **side-by-side** for clarity.
 - Real-world application (TrustTrainer app) dramatically increases the project's impact score.
-

★ Lesson 2 — Exploring the Dataset & Building a Fairness-Aware Baseline

1. Lesson Summary

In this lesson, the student will:

- Find out **what's actually inside the dataset** (utterances, speakers, demographics, labels).
- Learn how to **load and inspect** the data using pandas.
- Build a **simple "baseline" model** (majority-class or simple classifier) to measure starting performance.
- Compute **metrics by demographic group** (White, Black, South Asian) to see existing fairness gaps.

This is the bridge between the project plan and real code: by the end of Lesson 2, the student should be able to answer, with numbers, "How unfair is the starting point?"—a key question that ISEF judges will love.

2. Key Points

- Always **understand your dataset** before building complex models.
 - A **baseline** is a simple method you can beat (e.g., always predicting the majority class).
 - Metrics like **accuracy** alone can hide important fairness problems.
 - It's crucial to look at **per-group performance** (e.g., recall for South Asian speakers).
 - **pandas** is the main tool for exploring structured data.
 - We can simulate fairness analysis even on a **small toy dataset** before using the full OSF dataset.
 - Documenting early findings sets up your later "innovation" story.
-

3. Real-World Examples or Stories

- A hiring model that has **90% accuracy overall** might have **60% accuracy** for one demographic group. That's unacceptable in real life but invisible if you only look at the average.
- Facial recognition systems with high overall accuracy have historically misclassified **darker-skinned women** much more often than lighter-skinned men. Only **group-wise metrics** revealed this.
- For your project, the original Random Forest model had better recall for **White speakers** than for **South Asian speakers**, exposing WWIB (White Western Individualist Bias).

These stories motivate why Lesson 2 is not just "data cleaning"—it's about **justice** and **rigor**.

4. Terminology Explained

- **Dataset** – A collection of data points; here, audio utterances + labels + demographics.
 - **Label** – The target we want to predict; here: **trustworthy** vs **neutral**.
 - **Feature** – The input used by a model (later: Mel-spectrogram pixels).
 - **Exploratory Data Analysis (EDA)** – Systematic first look at data: shapes, counts, missing values, distributions.
 - **Baseline Model** – A simple model used as a reference point (e.g., majority-class predictor).
 - **Subgroup** – A subset of data defined by a shared attribute, like ethnicity.
 - **Accuracy** – Fraction of predictions that are correct.
 - **Recall** – Of all truly positive cases, how many the model correctly finds.
 - **Fairness Gap** – Difference in a metric (like recall) between demographic groups.
-

5. How It Works (Step-by-Step)

A. Load and Inspect the Data

1. Put the dataset (e.g., CSV with metadata) in a **data/** folder.
2. Use pandas to load:

```
import pandas as pd
df = pd.read_csv("data/metadata.csv")
print(df.head())
print(df.columns)
```

3. Check basic info:

```
print(df.shape)
print(df["ethnicity"].value_counts())
print(df["intent_label"].value_counts())
```

B. Build a Simple Baseline

- **Majority-class baseline:** Find the most common label, predict it for everyone.

```
majority_label = df["intent_label"].mode()[0]
df["baseline_pred"] = majority_label
```

- Or a **very simple classifier** (e.g., logistic regression on numeric features if available).

C. Compute Metrics Overall and by Group

```
from sklearn.metrics import accuracy_score, recall_score

acc = accuracy_score(df["intent_label"], df["baseline_pred"])
rec = recall_score(df["intent_label"], df["baseline_pred"],
pos_label="trustworthy")
print("Overall accuracy:", acc)
print("Overall recall (trustworthy):", rec)

for group, sub in df.groupby("ethnicity"):
    acc_g = accuracy_score(sub["intent_label"], sub["baseline_pred"])
    rec_g = recall_score(sub["intent_label"], sub["baseline_pred"],
pos_label="trustworthy")
    print(group, "Accuracy:", acc_g, "Recall:", rec_g)
```

D. Interpret Fairness Gaps

- If recall for **White** is 0.75 but for **SouthAsian** is 0.55 → **20% fairness gap**.
- This becomes your “**problem statement with numbers**” for your poster and paper.

6. Practice Exercises (5)

Exercise 1 – Column Inspection Given a CSV with columns: `utterance_id`, `speaker_id`, `ethnicity`, `intent_label`, write code to:

1. Load the file.
2. Print the first 5 rows.
3. Print the unique values in `ethnicity` and `intent_label`.

Exercise 2 – Distribution Counts Write code to:

1. Count how many rows there are for each ethnicity.
 2. Count how many **trustworthy** vs **neutral** labels there are overall.
-

Exercise 3 – Majority Baseline

1. Find the majority class for **intent_label**.
 2. Create a new column **baseline_pred** that always predicts this label.
-

Exercise 4 – Overall Metrics Use scikit-learn to compute:

- Overall accuracy
 - Overall recall for the **trustworthy** class
-

Exercise 5 – Group Metrics & Fairness Gap For each ethnicity:

1. Compute accuracy and recall for **baseline_pred**.
 2. Compute the **fairness gap** between the group with the highest recall and the lowest recall.
-

7. Solutions (Model Answers)

Solution 1 – Column Inspection

```
import pandas as pd

df = pd.read_csv("data/mini_trust_dataset.csv")
print(df.head())
print("Ethnicities:", df["ethnicity"].unique())
print("Intent labels:", df["intent_label"].unique())
```

Solution 2 – Distribution Counts

```
print("Counts per ethnicity:")
print(df["ethnicity"].value_counts())
print("\nLabel counts:")
print(df["intent_label"].value_counts())
```

Solution 3 – Majority Baseline

```
majority_label = df["intent_label"].mode()[0]
print("Majority label:", majority_label)
```

```
df["baseline_pred"] = majority_label
```

Solution 4 – Overall Metrics

```
from sklearn.metrics import accuracy_score, recall_score

acc = accuracy_score(df["intent_label"], df["baseline_pred"])
rec = recall_score(df["intent_label"], df["baseline_pred"],
pos_label="trustworthy")

print(f"Overall accuracy: {acc:.3f}")
print(f"Overall recall (trustworthy): {rec:.3f}")
```

Solution 5 – Group Metrics & Fairness Gap

```
group_recalls = {}

for group, sub in df.groupby("ethnicity"):
    acc_g = accuracy_score(sub["intent_label"], sub["baseline_pred"])
    rec_g = recall_score(sub["intent_label"], sub["baseline_pred"],
pos_label="trustworthy")
    group_recalls[group] = rec_g
    print(f"{group}: Accuracy={acc_g:.3f}, Recall(trustworthy)={rec_g:.3f}")

max_rec = max(group_recalls.values())
min_rec = min(group_recalls.values())
gap = max_rec - min_rec
print(f"\nFairness gap in recall: {gap:.3f}")
```

A gap > 0.10 (10 percentage points) is already a red flag the student can mention.

8. Q&A (10 Questions)

1. **Q:** Why do we bother with a simple baseline? **A:** To have a clear “starting line” to beat. Judges want to see improvement over something, not just one fancy model.
2. **Q:** Is a high overall accuracy always good? **A:** Not necessarily. If one group is large, it can dominate the metric and hide poor performance for others.
3. **Q:** Why use pandas instead of plain Python lists? **A:** pandas gives powerful tools (groupby, value_counts, etc.) that make analysis easier and less error-prone.
4. **Q:** Why group by ethnicity? **A:** Because bias often appears as performance differences between demographic groups.

5. **Q:** What if the dataset is imbalanced (e.g., more trustworthy than neutral)? **A:** Then a majority-class baseline might look strong on accuracy but can have poor recall for the minority class.
 6. **Q:** Should we always predict the majority class in real systems? **A:** No—it's only for benchmarking. Real systems must treat minority cases fairly.
 7. **Q:** Is recall the only fairness metric? **A:** No. Precision, F1, false positive rate, and other metrics can also matter, but recall is especially important when missing positives is harmful.
 8. **Q:** Could we use accuracy for fairness gap instead of recall? **A:** Yes, but recall is more sensitive when the positive class is rare or important (like detecting trustworthy intent).
 9. **Q:** Do I need the full OSF dataset to practice this lesson? **A:** No, we start with a small synthetic dataset (in the mini project) to learn the workflow.
 10. **Q:** How do these analyses appear in an ISEF poster? **A:** Typically as bar charts comparing recall/accuracy across groups, plus a short text explaining the fairness gap.
-

9. Quiz (10 Questions)

1. What is the purpose of a baseline model? Answer: To provide a simple reference level of performance that more advanced models should beat.

2. Which Python library is most useful for EDA on tabular data? Answer: `pandas`.

3. What is a majority-class baseline? Answer: A model that always predicts the most frequent label in the dataset.

4. Why is looking only at overall accuracy dangerous? Answer: It can hide poor performance on smaller demographic groups.

5. What function in pandas gives label counts? Answer: `value_counts()`.

6. What is a subgroup in fairness analysis? Answer: A subset of data defined by attributes like ethnicity, gender, or age.

7. Which metric answers: "Of all true trustworthy cases, how many did we catch?" Answer: Recall for the `trustworthy` class.

8. How do you compute fairness gap in recall? Answer: Max recall across groups minus min recall across groups.

9. What is one sign that your dataset may be biased? Answer: Large differences in performance metrics between demographic groups.

10. Why is documenting early metrics important? Answer: It lets you prove later that your new model genuinely reduces bias and improves over the baseline.

10. Mini Practice Project — Baseline Fairness Analysis (with .zip)

Goal

Use a **small synthetic dataset** to:

1. Load and inspect voice-intent data.
2. Build a majority-class baseline model.
3. Compute and interpret fairness metrics by ethnicity.

What's Included in the .zip

I've created a ready-to-use archive:

lesson2_miniproject_echoes_fairness.zip containing:

- **lesson2_miniproject_echoes/**
 - **README.md** – Instructions.
 - **data/mini_trust_dataset.csv** – Synthetic dataset with:
 - **utterance_id**
 - **speaker_id**
 - **ethnicity** (White, Black, SouthAsian)
 - **intent_label** (trustworthy, neutral)
 - **src/analyze_fairness.py** – Python script that:
 - Loads the dataset
 - Shows distributions
 - Builds a majority baseline
 - Computes accuracy & recall overall and by ethnicity

How to Run

1. Unzip the file.
2. (Optional but recommended) create a virtual environment.
3. Install dependencies:

```
pip install pandas scikit-learn
```

4. Run:

```
python src/analyze_fairness.py
```

5. Look at the printed metrics and answer:

- Which ethnicity has the highest recall for **trustworthy**?
- Which has the lowest?
- What is the fairness gap?
- How might this motivate your later CNN + adversarial debiasing work?

11. References

- Project plan PDF (Echoes of Equity) for definitions, milestones, and fairness goals.
- pandas documentation: <https://pandas.pydata.org/docs/>
- scikit-learn metrics guide: https://scikit-learn.org/stable/modules/model_evaluation.html
- Fairness in ML overview: <https://developers.google.com/machine-learning/fairness-overview>

12. Additional Information (Mentor Tips)

- Encourage the student to **copy plots and tables from this mini project** into their research notebook —this becomes early evidence of bias.
- Have them write a **short paragraph** summarizing their fairness findings; later this becomes part of the ISEF paper's "Baseline Results" section.
- Remind them: *"If you can clearly show the unfairness at the start, your later improvements will look much more impressive."*

★ Lesson 3 — Audio Processing & Mel-Spectrograms for Trustworthiness

1. Lesson Summary

This lesson teaches the student how to:

- Load **raw audio (.wav)** files.
- Understand basic audio concepts: **sampling rate, waveform, frequency**.
- Convert audio into **Mel-spectrograms**, which are image-like inputs for CNNs.
- Save and visualize spectrograms for later use in the deep learning model.

By the end, the student will be able to build a simple **audio → Mel-spectrogram pipeline**, a key building block for the project's CNN and adversarial debiasing architecture.

2. Key Points

- Deep learning models work best on **structured numeric grids** → spectrograms.
 - A **waveform** is sound amplitude over time; a **spectrogram** is energy over time & frequency.
 - The **Mel scale** is a frequency scale aligned with how humans perceive pitch.
 - **Short-Time Fourier Transform (STFT)** converts audio into time–frequency representation.
 - A **Mel-spectrogram** is a transformed STFT summed into Mel frequency bands.
 - Parameters like **sample rate, n_fft, hop_length, n_mels** affect resolution and model performance.
 - Libraries like **librosa** or **torchaudio** make audio processing manageable.
 - Properly saving and organizing spectrograms is essential for reproducible experiments.
-

3. Real-World Examples or Stories

- **Speech recognition (e.g., Google, Alexa)**: They feed spectrogram-like features into deep neural networks to recognize words and speakers.
- **Music genre classification**: Systems that guess if a track is rock, jazz, or classical almost always use spectrograms.
- **Birdsong recognition & bioacoustics**: Ecologists use spectrograms to detect specific bird calls or endangered species.

Your project does something similar but with an important twist: **predicting trustworthiness intent fairly across demographics**.

4. Terminology Explained

- **Sampling Rate (Hz)** – How many audio samples are taken per second (e.g., 16,000 Hz).
 - **Waveform** – A 1D array of audio amplitudes over time.
 - **Frequency (Hz)** – How fast a sound wave oscillates; higher frequency = higher pitch.
 - **STFT (Short-Time Fourier Transform)** – A sliding window FFT that converts small chunks of audio from time domain to frequency domain.
 - **Spectrogram** – A 2D matrix: time on x-axis, frequency on y-axis, intensity as color.
 - **Mel Scale** – A non-linear scale where equal steps sound equally spaced in pitch to humans.
 - **Mel-Spectrogram** – Spectrogram where frequencies are converted to the Mel scale.
 - **Decibels (dB)** – Logarithmic scale of intensity; good for viewing spectrograms.
 - **Log-Mel Spectrogram** – Mel-spectrogram converted to dB (log scale) for better visual and model performance.
-

5. How It Works (Step-by-Step)

A. Loading Audio

Using **librosa**:

```
import librosa

file_path = "data/example.wav"
y, sr = librosa.load(file_path, sr=16000) # y: waveform, sr: sample rate
```

```
print("Waveform length:", len(y))
print("Sample rate:", sr)
```

- `y` is a numpy array of floats between -1 and 1.
 - `sr` is the sampling rate (here we force 16 kHz for consistency).
-

B. Computing a Mel-Spectrogram

```
import librosa
import librosa.display
import matplotlib.pyplot as plt

n_fft = 1024      # FFT window size
hop_length = 256  # step between windows
n_mels = 64       # number of Mel bands

S = librosa.feature.melspectrogram(
    y=y,
    sr=sr,
    n_fft=n_fft,
    hop_length=hop_length,
    n_mels=n_mels,
    power=2.0
)

print("Mel-spectrogram shape:", S.shape)
```

- `S` has shape `(n_mels, time_frames)`.
 - Each row is a Mel band; each column is a time slice.
-

C. Converting to dB & Visualizing

```
S_db = librosa.power_to_db(S, ref=S.max())

plt.figure(figsize=(6,4))
librosa.display.specshow(
    S_db,
    sr=sr,
    hop_length=hop_length,
    x_axis="time",
    y_axis="mel"
)
plt.title("Mel-Spectrogram")
plt.colorbar(format="%+2.0f dB")
plt.tight_layout()
plt.show()
```

This image is what your CNN will see later.

D. Saving Spectrograms for CNN

Instead of plotting interactively, you can save to file:

```
plt.savefig("data/example_melspec.png")
```

Later, your CNN can either:

- Load these PNGs as images, or
 - Compute spectrograms on the fly directly from audio.
-

E. Connecting to the Project

In *Echoes of Equity*, the plan is to:

1. Download the 1,152 .wav files.
2. Convert each into a **Mel-spectrogram**.
3. Feed these spectrograms to a **2D-CNN** that predicts **trustworthy** vs **neutral**.
4. Add an **adversarial demographic head** later.

Lesson 3 sets up step 2 solidly.

6. Practice Exercises (5)

Exercise 1 – Load Audio & Print Info Write a script that:

1. Loads **demo_tone.wav** (or any speech file you have).
 2. Prints the sample rate and number of samples.
 3. Computes the duration in seconds.
-

Exercise 2 – Inspect Waveform Shape Using numpy, compute:

1. The minimum and maximum value of the waveform.
 2. The mean amplitude.
 3. A comment: does it look normalized (within [-1, 1])?
-

Exercise 3 – Mel-Spectrogram Parameters Compute a Mel-spectrogram with:

- **n_fft=1024, hop_length=256, n_mels=64**

Then try **n_mels=128**. Compare shapes and write down how the image changes.

Exercise 4 – Save Spectrogram Image Modify your code to:

1. Save the Mel-spectrogram to `data/example_melspec.png`.
2. Open the image to visually inspect it.

Exercise 5 – Time-Frequency Tradeoff Change `hop_length` from 256 to 512 and 128. For each version, note:

- How does the time resolution change?
 - How does the visual look differ (more squished or stretched in time)?
-

7. Solutions (Model Answers)

Solution 1 – Load Audio & Print Info

```
import librosa

y, sr = librosa.load("data/demo_tone.wav", sr=None)
print("Sample rate:", sr)
print("Number of samples:", len(y))

duration = len(y) / sr
print("Duration (sec):", duration)
```

Solution 2 – Inspect Waveform Shape

```
import numpy as np
import librosa

y, sr = librosa.load("data/demo_tone.wav", sr=None)
print("Min:", y.min())
print("Max:", y.max())
print("Mean:", y.mean())
```

Typically, `y.min()` and `y.max()` should be between -1 and 1; `mean` close to 0.

Solution 3 – Mel-Spectrogram Parameters

```
import librosa

y, sr = librosa.load("data/demo_tone.wav", sr=None)

S1 = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=1024, hop_length=256,
n_mels=64)
S2 = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=1024, hop_length=256,
n_mels=128)
```

```
print("Shape with 64 mels:", S1.shape) # (64, T)
print("Shape with 128 mels:", S2.shape) # (128, T)
```

Explanation: More Mel bands → higher “vertical” resolution.

Solution 4 – Save Spectrogram Image

```
import librosa
import librosa.display
import matplotlib.pyplot as plt

y, sr = librosa.load("data/demo_tone.wav", sr=None)
S = librosa.feature.melspectrogram(y=y, sr=sr, n_fft=1024, hop_length=256,
n_mels=64)
S_db = librosa.power_to_db(S, ref=S.max())

plt.figure(figsize=(6,4))
librosa.display.specshow(S_db, sr=sr, hop_length=256, x_axis="time", y_axis="mel")
plt.title("Mel-Spectrogram")
plt.colorbar(format="%+2.0f dB")
plt.tight_layout()
plt.savefig("data/demo_tone_melspec.png")
print("Saved to data/demo_tone_melspec.png")
```

Solution 5 – Time-Frequency Tradeoff

Key idea:

- **Smaller hop_length** (e.g., 128) → more time frames → better time resolution, heavier computation.
- **Larger hop_length** (e.g., 512) → fewer time frames → coarser time resolution, lighter computation.

Students should describe how the spectrogram looks more stretched/compressed along the time axis.

8. Q&A (10 Frequently Asked Questions)

1. **Q:** Why Mel-spectrograms instead of raw waveforms? **A:** They encode frequency and time information in a way that aligns with human perception and are easier for CNNs to learn from.
2. **Q:** Can I change the sample rate? **A:** Yes, but keep it consistent across all samples (e.g., 16 kHz) to avoid mismatched feature shapes.
3. **Q:** What does `n_fft` do? **A:** It controls the window size of the STFT; larger `n_fft` → better frequency resolution, but more computation.
4. **Q:** Why convert to dB (`power_to_db`)? **A:** Log scale (dB) better reflects human hearing and spreads out low-intensity differences.

5. **Q:** What happens if `n_mels` is very small (e.g., 20)? **A:** You lose fine-grained frequency detail, which can hurt classification.
 6. **Q:** Why do we need `hop_length`? **A:** It controls how much we slide the window; smaller hop = more overlapping windows = smoother but heavier.
 7. **Q:** Is the color in the spectrogram important for the CNN? **A:** The CNN sees numbers, not “colors”; colormap is just for humans. The numeric values are what matter.
 8. **Q:** Can I use torchaudio instead of librosa? **A:** Yes, especially if your training is in PyTorch; concepts are identical.
 9. **Q:** Should I save spectrogram as PNG or as a numpy array? **A:** Either is fine; PNG is convenient for visualization, numpy arrays can be more precise and efficient for training.
 10. **Q:** How does this relate to fairness? **A:** The same processing pipeline is applied to all speakers; later, bias comes from modeling and data distribution, not from unequal preprocessing.
-

9. Quiz (10 Questions)

1. **What does a waveform represent?** → Amplitude of sound over time.
 2. **What is the Mel scale?** → A perceptual frequency scale aligned with human hearing.
 3. **What function in librosa computes Mel-spectrograms?** → `librosa.feature.melspectrogram`.
 4. **What parameter controls the number of Mel frequency bands?** → `n_mels`.
 5. **Increasing `n_mels` does what to the spectrogram?** → Increases vertical (frequency) resolution.
 6. **What does STFT stand for?** → Short-Time Fourier Transform.
 7. **Why convert power spectrogram to dB?** → To use a log scale like human perception and enhance low-energy details.
 8. **What controls time resolution in a spectrogram?** → `hop_length`.
 9. **If you double `hop_length`, what happens to the number of time frames?** → Roughly halves (coarser time resolution).
 10. **Why are spectrograms useful for CNNs?** → They transform audio into 2D grids (like images) that CNNs are good at processing.
-

10. Mini Practice Project – WAV → Mel-Spectrogram (with .zip)

I've created a ready-made mini project so the student can practice the full audio pipeline.

Project Goal

- Load a simple audio file (`demo_tone.wav`).
- Compute its Mel-spectrogram with librosa.

- Save a PNG image of the spectrogram.

What's in the .zip

`lesson3_miniproject_melspec.zip` contains:

- `lesson3_miniproject_melspec/`
 - `README.md` – Setup and run instructions.
 - `data/demo_tone.wav` – A generated sine-wave audio file.
 - `src/make_melspec.py` – Script that:
 - Loads `demo_tone.wav`
 - Computes a Mel-spectrogram
 - Converts to dB
 - Saves it as `data/demo_tone_melspec.png`

How to Use

1. Unzip the file.
2. In a terminal inside the unzipped folder:

```
pip install librosa matplotlib
python src/make_melspec.py
```

3. Open `data/demo_tone_melspec.png` and inspect the result.
4. Bonus: replace `demo_tone.wav` with your own short speech recording and rerun.

11. References

- Project description and milestones for audio processing and Mel-spectrograms.
- Librosa documentation (audio & spectrograms): <https://librosa.org/doc/latest/>
- Intro video on spectrograms (any YouTube “What is a spectrogram?” works well).
- PyTorch Audio Tutorial (if later using torchaudio):
https://pytorch.org/tutorials/beginner/audio_preprocessing_tutorial.html

12. Additional Information (Competition-Focused Tips)

- Encourage the student to **save a few example spectrograms** from different speakers and intents. These can become **figures on the poster** showing qualitative differences.
- Have them note in their logbook which **parameters** they used (`n_fft`, `hop_length`, `n_mels`)—later they can mention a brief **hyperparameter exploration** section.
- A nice ISEF-style figure:

- Panel A: waveform
- Panel B: Mel-spectrogram (neutral)
- Panel C: Mel-spectrogram (trustworthy) With captions discussing visible differences.

★ Lesson 4 — Building Your First CNN for Trustworthiness Classification

1. Lesson Summary

In this lesson, the student learns how to:

- Treat Mel-spectrograms as **image-like inputs**.
- Build a small **Convolutional Neural Network (CNN)** in PyTorch.
- Train the CNN on a simple dataset and monitor **training/validation accuracy**.
- Understand the basic building blocks: convolution, ReLU, pooling, flatten, dense layers.

This CNN is a **non-adversarial baseline**: later, you will modify and extend it with an adversarial head to reduce demographic bias as described in the project plan.

2. Key Points

- CNNs are ideal for **2D grid data** like images and spectrograms.
 - A CNN learns **local patterns** (edges, shapes, textures) with convolution filters.
 - **Pooling** layers reduce spatial size, making the model faster and more robust.
 - **ReLU** introduces nonlinearity so the network can learn complex relationships.
 - A typical architecture for spectrograms: Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → Dense → Output.
 - We split data into **train** and **validation** sets to measure generalization.
 - We use a **loss function** (CrossEntropyLoss) and an **optimizer** (Adam/SGD) to update weights.
 - This simple CNN becomes the “baseline deep model” that your fairness-aware model will later improve upon.
-

3. Real-World Examples or Stories

- **Speech command recognition (e.g., “OK Google”)** uses CNN-like models on short spectrogram snippets.
- **Environmental sound classification** (sirens, dog barking, door closing) relies heavily on CNNs on audio spectrograms.
- **Emotion recognition from voice**: CNNs learn patterns of prosody associated with emotions like anger or joy.

Your project is similar: a CNN on spectrograms learns acoustic patterns associated with “trustworthy” vs “neutral” intent.

4. Terminology Explained

- **Convolutional Layer (Conv2d)** – A layer that slides small filters across the input to detect local patterns (like edges, frequency bands, etc.).
 - **Filter / Kernel** – A small matrix (e.g., 3×3) that is multiplied with local patches of the input.
 - **Feature Map** – The output of one filter; each filter learns to detect a specific pattern.
 - **ReLU (Rectified Linear Unit)** – Activation function $f(x) = \max(0, x)$ that introduces nonlinearity.
 - **Max Pooling** – Operation that takes the maximum value in a small window (e.g., 2×2), reducing resolution while keeping important features.
 - **Flatten** – Reshapes 2D feature maps into a 1D vector so it can be fed to fully connected layers.
 - **Fully Connected (Dense) Layer** – Classic neural layer where each input connects to each output.
 - **Epoch** – One full pass through the training dataset.
 - **Batch** – A subset of the dataset processed in one gradient update.
 - **Loss Function** – Measures how wrong the model is; training aims to minimize it.
-

5. How It Works (Step-by-Step)

Here's a conceptual pipeline for this lesson (with synthetic data for now):

1. Prepare input data

- Use 2D spectrograms (or synthetic "spectrogram-like" arrays) of shape (1, H, W).
- Labels: 0 for neutral, 1 for trustworthy.

2. Create a PyTorch Dataset

```
from torch.utils.data import Dataset

class SpectrogramDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.from_numpy(X) # shape: (N, 1, H, W)
        self.y = torch.from_numpy(y) # shape: (N,)

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

3. Build a SimpleCNN

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
```

```

        nn.ReLU(),
        nn.MaxPool2d(2),          # halves H and W
        nn.Conv2d(8, 16, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(2),
        nn.Flatten(),
        nn.Linear(16 * 8 * 8, 32),
        nn.ReLU(),
        nn.Linear(32, 2)          # 2 classes
    )

    def forward(self, x):
        return self.net(x)

```

4. Training Loop

- Use **CrossEntropyLoss** to compare predictions vs labels.
- Use **Adam** optimizer to update weights.
- For each epoch:
 - Loop over batches, compute loss, backprop, optimizer step.
 - Track training accuracy and loss.
 - Evaluate on validation set after each epoch.

5. Result

You should see validation accuracy go above 0.5 (better than chance). This proves the CNN can learn patterns from spectrogram-like data. Later you'll swap in real Mel-spectrograms from the trustworthiness dataset.

6. Practice Exercises (5)

Exercise 1 – CNN Architecture Sketch On paper or in a markdown cell, sketch a tiny CNN architecture for spectrograms of size **1×32×32**:

- Two conv layers, two max-pool layers, one hidden dense layer, one output layer. Write down the shape of the data after each layer.

Exercise 2 – Dataset Class Write a PyTorch **Dataset** class that:

- Accepts numpy arrays **X** ($N \times 1 \times 32 \times 32$) and **y** (N).
- Returns (**spectrogram**, **label**) pairs.

Exercise 3 – DataLoader Setup Using your **Dataset**, create a PyTorch **DataLoader** for:

- Batch size = 16
- Shuffle = True for training set

Exercise 4 – Forward Pass Test Instantiate `SimpleCNN`, create a fake batch of data with shape `(4, 1, 32, 32)`, and:

- Run it through the model.
 - Print the output shape.
-

Exercise 5 – Training Epoch Skeleton Write code to perform **one epoch** of training:

- Loop over `train_loader`.
 - Compute logits, loss, backprop, and optimizer step.
 - Track batch accuracy and print average at the end.
-

7. Solutions (Model Answers)

Solution 1 – CNN Architecture Sketch (Shapes) Input: `(1, 32, 32)`

- Conv1 (1→8, kernel 3, padding 1): `(8, 32, 32)`
 - MaxPool1 (2×2): `(8, 16, 16)`
 - Conv2 (8→16, kernel 3, padding 1): `(16, 16, 16)`
 - MaxPool2 (2×2): `(16, 8, 8)`
 - Flatten: $16 * 8 * 8 = 1024$
 - Dense1: $1024 \rightarrow 32$
 - Dense2: $32 \rightarrow 2$
-

Solution 2 – Dataset Class

```
from torch.utils.data import Dataset
import torch

class SpectrogramDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.from_numpy(X)      # float32
        self.y = torch.from_numpy(y)      # int64

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

Solution 3 – DataLoader Setup

```
from torch.utils.data import DataLoader

train_ds = SpectrogramDataset(X_train, y_train)
```

```
val_ds = SpectrogramDataset(X_val, y_val)

train_loader = DataLoader(train_ds, batch_size=16, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=16, shuffle=False)
```

Solution 4 – Forward Pass Test

```
import torch

model = SimpleCNN()
fake_batch = torch.randn(4, 1, 32, 32) # 4 spectrograms
logits = model(fake_batch)
print("Output shape:", logits.shape) # should be (4, 2)
```

Solution 5 – Training Epoch Skeleton

```
import torch.nn as nn
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

model.train()
running_loss = 0.0
correct = 0
total = 0

for Xb, yb in train_loader:
    Xb, yb = Xb.to(device), yb.to(device)

    optimizer.zero_grad()
    logits = model(Xb)
    loss = criterion(logits, yb)
    loss.backward()
    optimizer.step()

    running_loss += loss.item() * Xb.size(0)
    preds = logits.argmax(dim=1)
    correct += (preds == yb).sum().item()
    total += yb.size(0)

epoch_loss = running_loss / total
epoch_acc = correct / total
print(f"Train loss: {epoch_loss:.4f}, Train acc: {epoch_acc:.3f}")
```

8. Q&A (10 Common Questions)

1. **Q:** Why use a CNN instead of a regular dense network on flattened pixels? **A:** CNNs exploit local structure and share weights, making them more efficient and better at learning spatial patterns.
2. **Q:** How many layers should my CNN have? **A:** Start small (2 conv layers) so it's easy to debug; you can add more later if needed.
3. **Q:** What if my model overfits? **A:** Use regularization (dropout, weight decay), data augmentation, or smaller models.
4. **Q:** Should I use ReLU or another activation? **A:** ReLU is standard and works well. You can experiment with LeakyReLU or others later.
5. **Q:** Why do we split into train and validation sets? **A:** To measure how well the model generalizes to unseen data.
6. **Q:** What is CrossEntropyLoss? **A:** A loss function for multi-class classification that compares predicted probabilities to true class labels.
7. **Q:** What does the optimizer do? **A:** It updates model weights using gradients from backpropagation to reduce the loss.
8. **Q:** How do I know if I'm using the GPU? **A:** Check `torch.cuda.is_available()` and move model and tensors to `device`.
9. **Q:** What is a "good" validation accuracy? **A:** For synthetic data, > 0.9 is possible; for real data, you compare against your baseline (e.g., > 71% from the Random Forest in the paper).
10. **Q:** When do we bring in fairness/adversarial training? **A:** After we have a strong CNN baseline; this lesson is about building that foundation.

9. Quiz (10 Questions)

1. **What type of data are spectrograms?** → 2D grid data (like images).
2. **Which layer in a CNN detects local patterns?** → Convolutional layer (Conv2d).
3. **What does ReLU stand for, and what is its formula?** → Rectified Linear Unit; $f(x) = \max(0, x)$.
4. **Why do we use MaxPool2d?** → To reduce spatial size and focus on the most important features.
5. **After two 2×2 max-pooling layers, how much are H and W reduced?** → Reduced by a factor of 4 (half twice).
6. **Which loss function is standard for multi-class classification in PyTorch?** → `nn.CrossEntropyLoss`.
7. **What does `.argmax(dim=1)` do on model outputs?** → Picks the index of the largest logit, i.e., the predicted class.

8. **Why do we shuffle the training data each epoch?** → To reduce learning order bias and improve generalization.
 9. **What is an epoch?** → One full pass through the entire training dataset.
 10. **How do you move a model to GPU in PyTorch?** → `model.to(device)` where `device = torch.device("cuda" if torch.cuda.is_available() else "cpu")`.
-

10. Mini Practice Project – Simple CNN on Synthetic Spectrograms (with .zip)

To make this very hands-on, I've prepared a **mini project** that trains a CNN on synthetic spectrogram-like data.

Project Goal

- Load a synthetic dataset of (1, 32, 32) "spectrograms" with 2 classes.
- Train a small CNN in PyTorch.
- Observe training and validation accuracy over 10 epochs.

What's in the .zip

`lesson4_miniproject_cnn.zip` contains:

- `lesson4_miniproject_cnn/`
 - `README.md` – Instructions.
 - `data/synthetic_spectrograms.npz` – Numpy archive with:
 - `X`: (120, 1, 32, 32) float32 arrays
 - `y`: (120,) int64 labels (0 = neutral, 1 = trustworthy)
 - `src/train_cnn.py` – Script that:
 - Loads the dataset
 - Splits into train/validation sets
 - Defines `SpectrogramDataset` and `SimpleCNN`
 - Trains for 10 epochs and prints train/val accuracy

How to Run

1. Unzip the archive.
2. Create a Python environment and install dependencies:

```
pip install torch torchvision numpy scikit-learn
```

3. Run:

```
python src/train_cnn.py
```

4. Watch the printed training/validation accuracy.
5. Optional: modify the architecture (more filters, different hidden size) and see how performance changes.

11. References

- Project plan sections on **Model Innovation – Train CNN** and **Deep Learning Model Training**.
- PyTorch Tutorials: <https://pytorch.org/tutorials/>
- “Convolutional Neural Networks” chapter in *Dive into Deep Learning* (free online textbook).
- Any intro CNN explanation videos (e.g., 3Blue1Brown’s neural network series).

12. Additional Information (Coach Notes for ISEF)

- Have the student treat this mini CNN as a **sandbox**: try different `n_filters`, kernel sizes, and compare metrics.
- Encourage them to document **hyperparameters** and results in a table—this becomes part of the “Model Architecture & Hyperparameter Tuning” section in the paper.
- Later, when they move from synthetic data to real Mel-spectrograms from the OSF dataset, they’ll reuse the **same code structure**, just changing the data loader.

★ Lesson 5 — Evaluation Metrics, Confusion Matrices & Fairness Gaps

1. Lesson Summary

In this lesson, the student will:

- Learn the main **classification metrics**: accuracy, precision, recall, F1, ROC, AUC.
- Understand how to build and interpret a **confusion matrix**.
- Compute metrics **per demographic group** and measure **fairness gaps**.
- Connect metrics to the project goal: **reducing bias** across White, Black, and South Asian speakers.

By the end, the student will be able to clearly explain, with tables and plots, how well a model performs overall *and* how fair it is.

2. Key Points

- Accuracy alone can be misleading, especially with **imbalanced data**.
- **Confusion matrices** reveal the types of errors a model makes.

- **Precision** and **recall** describe different aspects of performance.
 - **F1-score** balances precision and recall.
 - **ROC curve** and **AUC** show performance at all thresholds, not just 0.5.
 - Per-group metrics (by ethnicity) are essential for evaluating **demographic bias**.
 - **Fairness gap** = difference in a metric (e.g., recall) between the best and worst group.
 - Clear visualizations (tables, ROC plots, bar charts) are crucial for ISEF-level communication.
-

3. Real-World Examples or Stories

- A healthcare AI might have **95% accuracy overall**, but if recall for a minority group is only **70%**, that group is seriously underserved.
- The original trustworthiness paper reports different recall values for **White vs South Asian speakers**; your project's goal is to **close that gap**, not just bump up overall accuracy.
- Face recognition and recidivism prediction systems became controversial precisely because subgroup metrics revealed **large fairness gaps**.

Your student's project will stand out at ISEF if they show *both* overall performance and careful fairness analysis.

4. Terminology Explained

- **True Positive (TP)** – Model predicts “trustworthy” and it is truly trustworthy.
 - **True Negative (TN)** – Model predicts “neutral” and it is truly neutral.
 - **False Positive (FP)** – Model predicts “trustworthy” but it's actually neutral.
 - **False Negative (FN)** – Model predicts “neutral” but it's actually trustworthy.
 - **Confusion Matrix** – A 2×2 table of TP, FP, FN, TN for binary classification.
 - **Accuracy** – $(TP + TN) / (TP + TN + FP + FN)$.
 - **Precision (Trustworthy)** – $TP / (TP + FP)$ → “Of what I predicted as trustworthy, how many were correct?”
 - **Recall (Trustworthy)** – $TP / (TP + FN)$ → “Of all truly trustworthy, how many did I catch?”
 - **F1 Score** – Harmonic mean of precision and recall.
 - **ROC Curve** – Plot of True Positive Rate vs False Positive Rate across thresholds.
 - **AUC (Area Under the ROC Curve)** – Single number summarizing ROC performance (1.0 = perfect, 0.5 = random).
 - **Fairness Gap** – Difference between best and worst subgroup metric (e.g., recall).
-

5. How It Works (Step-by-Step)

Assume you have:

- **y_true** – true labels (0 = neutral, 1 = trustworthy).
- **y_pred** – predictions at threshold 0.5.
- **score_trustworthy** – predicted probability/logit for “trustworthy”.
- **ethnicity** – **White**, **Black**, **SouthAsian**.

A. Overall Confusion Matrix & Metrics

```
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc

cm = confusion_matrix(y_true, y_pred)
print(cm) # [[TN, FP],
           # [FN, TP]]

print(classification_report(y_true, y_pred, target_names=
["neutral", "trustworthy"]))
```

B. ROC Curve & AUC

```
fpr, tpr, thresholds = roc_curve(y_true, score_trustworthy)
roc_auc = auc(fpr, tpr)
print("AUC:", roc_auc)
```

Plot with matplotlib to visualize trade-offs.

C. Metrics by Ethnicity

```
import pandas as pd
from sklearn.metrics import recall_score, precision_score, f1_score

df = pd.DataFrame({
    "y_true": y_true,
    "y_pred": y_pred,
    "score_trustworthy": score_trustworthy,
    "ethnicity": ethnicity
})

group_recalls = {}
for group, sub in df.groupby("ethnicity"):
    rec = recall_score(sub["y_true"], sub["y_pred"], pos_label=1)
    prec = precision_score(sub["y_true"], sub["y_pred"], pos_label=1)
    f1 = f1_score(sub["y_true"], sub["y_pred"], pos_label=1)
    group_recalls[group] = rec
    print(group, "Precision:", prec, "Recall:", rec, "F1:", f1)
```

D. Fairness Gap

```
max_rec = max(group_recalls.values())
min_rec = min(group_recalls.values())
gap = max_rec - min_rec
print("Fairness gap in recall:", gap)
```

The student can later compare **Random Forest vs CNN vs Adversarial CNN** using these metrics, showing how fairness improves.

6. Practice Exercises (5)

Exercise 1 – Manually Compute Metrics from a Confusion Matrix Given a confusion matrix:

$\begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}$

$\begin{bmatrix} 40 & 10 \\ 5 & 45 \end{bmatrix}$

Compute accuracy, precision (trustworthy), recall (trustworthy), and F1.

Exercise 2 – Confusion Matrix in Python Write code using scikit-learn to print a confusion matrix and classification report for given `y_true` and `y_pred`.

Exercise 3 – ROC & AUC Given arrays `y_true` and `score_trustworthy`, use scikit-learn to:

1. Compute `fpr`, `tpr`, `thresholds`.
 2. Compute `auc(fpr, tpr)`.
 3. Plot the ROC curve.
-

Exercise 4 – Per-Group Recall Using a DataFrame with `y_true`, `y_pred`, and `ethnicity`, calculate recall for each group and print them.

Exercise 5 – Fairness Gap & Interpretation Using the per-group recalls from Exercise 4:

1. Compute the fairness gap.
 2. Write 3–4 sentences interpreting what this gap means and how it might affect real users.
-

7. Solutions (Model Answers)

Solution 1 – Manual Metrics

From:

- $\text{TN} = 40, \text{FP} = 10, \text{FN} = 5, \text{TP} = 45$

Total = $40 + 10 + 5 + 45 = 100$

- Accuracy = $(\text{TP} + \text{TN}) / \text{Total} = (45 + 40) / 100 = \mathbf{0.85}$
- Precision (trustworthy) = $\text{TP} / (\text{TP} + \text{FP}) = 45 / (45 + 10) = 45/55 \approx \mathbf{0.818}$
- Recall (trustworthy) = $\text{TP} / (\text{TP} + \text{FN}) = 45 / (45 + 5) = 45/50 = \mathbf{0.90}$

- $F1 = 2 * (P * R) / (P + R) \approx 2 * (0.818 * 0.90) / (0.818 + 0.90) \approx \mathbf{0.857}$

Solution 2 – Confusion Matrix in Python

```
from sklearn.metrics import confusion_matrix, classification_report

cm = confusion_matrix(y_true, y_pred)
print("Confusion matrix:\n", cm)
print("\nClassification report:")
print(classification_report(y_true, y_pred, target_names=["neutral",
"trustworthy"]))
```

Solution 3 – ROC & AUC

```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_true, score_trustworthy)
roc_auc = auc(fpr, tpr)
print("AUC:", roc_auc)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.2f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

Solution 4 – Per-Group Recall

```
import pandas as pd
from sklearn.metrics import recall_score

df = pd.DataFrame({"y_true": y_true, "y_pred": y_pred, "ethnicity": ethnicity})

for group, sub in df.groupby("ethnicity"):
    rec = recall_score(sub["y_true"], sub["y_pred"], pos_label=1)
    print(group, "Recall(trustworthy):", rec)
```

Solution 5 – Fairness Gap & Interpretation

```

recalls = {}
for group, sub in df.groupby("ethnicity"):
    rec = recall_score(sub["y_true"], sub["y_pred"], pos_label=1)
    recalls[group] = rec

max_rec = max(recalls.values())
min_rec = min(recalls.values())
gap = max_rec - min_rec
print("Fairness gap:", gap)

```

Interpretation example:

If recall is 0.85 for White speakers and 0.70 for South Asian speakers, the gap is 0.15 (15 percentage points). That means South Asian speakers are 15% more likely to have their trustworthy speech missed by the model. In real-world use (like public speaking coaching or hiring), this could systematically disadvantage them.

8. Q&A (10 Common Questions)

1. **Q:** Why isn't accuracy enough? **A:** Because it hides how errors are distributed. A model could be very accurate overall but still perform poorly on a minority group.
2. **Q:** When is recall more important than precision? **A:** When **missing positives** is worse than having some false alarms—for example, failing to detect trustworthy speech for certain demographics.
3. **Q:** What does a point on the ROC curve represent? **A:** The (FPR, TPR) pair at a specific decision threshold.
4. **Q:** Can AUC be high even if fairness is bad? **A:** Yes. A model might rank samples well overall but still treat some subgroups worse.
5. **Q:** What threshold should I use? **A:** 0.5 is common, but you can tune it based on the trade-off between precision and recall. You can also compare subgroup performance at your chosen threshold.
6. **Q:** Why focus on recall for "trustworthy" rather than "neutral"? **A:** The project's risk is missing people who are actually trustworthy (false negatives), which can harm minority speakers more.
7. **Q:** What is macro vs weighted F1? **A:** Macro F1 averages F1 over classes equally; weighted F1 weights by class frequency.
8. **Q:** How do judges like to see metrics presented? **A:** Confusion matrices, metric tables, and ROC curves, plus clear explanations of what improved and how fairness gaps changed.
9. **Q:** Can fairness gap ever be 0? **A:** In theory yes, but in practice small gaps are fine; your goal is to **reduce** large gaps from the baseline.
10. **Q:** How does this connect to adversarial debiasing later? **A:** These metrics are how you **prove** that adversarially trained models reduce subgroup gaps compared to non-debiased models.

9. Quiz (10 Questions)

1. **Which of the following is TP?** A) Model predicts neutral, true label neutral B) Model predicts trustworthy, true label trustworthy ☒ C) Model predicts neutral, true label trustworthy D) Model predicts trustworthy, true label neutral
 2. **Accuracy formula:** $\rightarrow (TP + TN) / (TP + TN + FP + FN)$
 3. **Recall (trustworthy) measures:** $\rightarrow$ Of all truly trustworthy samples, how many the model correctly labels as trustworthy.
 4. **Precision (trustworthy) measures:** $\rightarrow$ Of all samples predicted trustworthy, how many are truly trustworthy.
 5. **What metric balances precision and recall?** $\rightarrow$ F1-score.
 6. **On an ROC curve, the x-axis is:** $\rightarrow$ False Positive Rate.
 7. **AUC of 0.5 means:** $\rightarrow$ The model is performing like random guessing.
 8. **A fairness gap in recall of 0.20 means:** $\rightarrow$ The best group's recall is 20 percentage points higher than the worst group's.
 9. **Which function computes an ROC curve in scikit-learn?** $\rightarrow$ `roc_curve`.
 10. **Why compute metrics per ethnicity?** $\rightarrow$ To detect and measure demographic bias in the model.
-

10. Mini Practice Project – Evaluation & Fairness Metrics (with .zip)

I prepared a mini project so the student can *actually* compute these metrics on a small synthetic dataset.

Project Goal

- Load a small CSV with **true labels**, **predicted scores**, **hard predictions**, and **ethnicity**.
- Compute confusion matrix, precision, recall, F1, ROC curve, AUC.
- Compute per-ethnicity metrics and fairness gap in recall.
- Save the ROC curve as an image.

What's in the .zip

`lesson5_miniproject_eval.zip` contains:

- `lesson5_miniproject_eval/`
 - `README.md` – Instructions.
 - `data/synthetic_eval_results.csv` – Columns:
 - `y_true` (0/1)
 - `score_trustworthy` (0–1)
 - `y_pred` (0/1 at threshold 0.5)
 - `ethnicity` (White, Black, SouthAsian)

- `src/analyze_metrics.py` – Script that:
 - Loads the CSV
 - Prints confusion matrix & classification report
 - Computes ROC & AUC and saves `data/roc_overall.png`
 - Computes per-ethnicity precision/recall/F1
 - Prints fairness gap in recall

How to Run

1. Unzip the archive.
2. Install dependencies:

```
pip install pandas numpy scikit-learn matplotlib
```

3. Run:

```
python src/analyze_metrics.py
```

4. Check the printed metrics and open `data/roc_overall.png`.
5. Have the student write a **short paragraph** summarizing:
 - Overall accuracy & AUC
 - Recall per group
 - Fairness gap and what it means

This paragraph can later be adapted directly into the ISEF paper's "**Baseline Evaluation**" section.

11. References

- Project timeline sections: "Replication – Confusion Matrix" and "Final Analysis & Presentation."
 - scikit-learn metrics documentation: https://scikit-learn.org/stable/modules/model_evaluation.html
 - ROC & AUC explanation: any standard ML blog/tutorial (e.g., scikit-learn's ROC tutorial).
 - Papers and tutorials on fairness metrics (demographic parity, equalized odds) for further reading.
-

12. Additional Information (Mentor Tips)

- Encourage the student to **store all confusion matrices and ROC curves** for each model version (Random Forest, CNN, adversarial CNN). This history becomes powerful evidence of improvement.
- For the final poster, I'd suggest:
 - One **overall ROC curve** comparing different models.
 - A **bar chart** of recall per ethnicity for baseline vs debiased model.

- Emphasize that *clear evaluation* is what differentiates a “cool project” from a **publishable, competition-winning study**.
-

★ Lesson 6 — Adversarial Debiasing & Gradient Reversal for Fair Representations

1. Lesson Summary

In this lesson, the student will:

- Understand the idea of **adversarial debiasing**: training a model that
 - predicts **trustworthiness intent** well, while
 - **unlearning demographic cues** (ethnicity).
- Learn how a **Gradient Reversal Layer (GRL)** works conceptually.
- See how to build a network with:
 - **Shared feature extractor** (from Mel-spectrograms),
 - **Main head** (trustworthy vs neutral),
 - **Adversarial head** (ethnicity).
- Train on a synthetic dataset and observe how increasing GRL strength lowers **group-prediction accuracy** (good) while keeping **main-task accuracy** reasonable.

This lesson directly corresponds to the “adversarial head” and “reduce WWIB bias” design in your project plan.

2. Key Points

- Standard models may unintentionally encode **demographic information** in their internal representations.
- Adversarial debiasing adds an extra network that tries to **predict the sensitive attribute** (ethnicity), while the feature extractor tries to **fool** it.
- The **Gradient Reversal Layer (GRL)** multiplies gradients by $-\lambda$, turning a minimization objective into a **maximization** from the feature extractor’s perspective.
- The main task (trustworthy vs neutral) is trained **normally**, so we still want high performance.
- The adversarial head being **bad** at group prediction is a sign of **less encoded bias**.
- λ (lambda) controls how strongly we push the model to forget demographic information.
- We evaluate success by checking:
 - Main accuracy / recall, and

- Group (ethnicity) accuracy from the adversarial head and fairness gaps.

3. Real-World Examples or Stories

- In hiring algorithms, we do not want internal features to strongly predict **race or gender**, even if those attributes are not explicitly in the input—because that leads to systemic discrimination.
- In your project, the goal is to build a trustworthiness model whose internal features **do not encode ethnicity**, reducing WWIB (White Western Individualist Bias).
- Think of the adversarial head as a “mini-judge” trying to guess ethnicity from features; the feature extractor learns to “hide” ethnicity from that judge while still keeping signal for trustworthiness.

4. Terminology Explained

- **Sensitive Attribute** – A personal characteristic like ethnicity, gender, or age that we want the model to treat fairly.
- **Adversarial Head** – A classifier that predicts the sensitive attribute using the shared features.
- **Gradient Reversal Layer (GRL)** – A special layer that passes data unchanged in the forward pass but multiplies gradients by $-\lambda$ in the backward pass.
- **λ (lambda)** – A hyperparameter controlling how strongly we push the model to remove sensitive information.
- **Shared Feature Extractor** – Network part that turns input (spectrogram) into a compact representation used by both heads.
- **Domain Adversarial Training** – General technique similar to what we’re doing; often used to make models robust across domains (e.g., different accents).
- **Representation Invariance** – The property that features are not predictive of certain attributes (like demographic group).

5. How It Works (Step-by-Step)

A. Architecture Layout

We split the model into three pieces:

1. **FeatureExtractor (F)**: Takes spectrogram (or synthetic feature vector) → outputs **h** (hidden representation).
2. **MainHead (M)**: Takes **h** → predicts **y_main** (trustworthy vs neutral).
3. **GroupHead (G)**: Takes **gradient-reversed h** → predicts **y_group** (ethnicity).

Formally:

- Forward pass:
 - $h = F(x)$
 - $y_main_logits = M(h)$
 - $h_rev = GRL(h)$
 - $y_group_logits = G(h_rev)$

- Loss:
 - $L_{total} = L_{main}(y_{main_logits}, y_{main}) + L_{group}(y_{group_logits}, y_{group})$

Because of GRL, F is pushed to **minimize** L_{main} but **maximize** L_{group} .

B. Gradient Reversal Layer (GRL)

Conceptually:

- Forward: $GRL(h) = h$ (do nothing).
- Backward: $\partial L / \partial h = -\lambda * \partial L / \partial (GRL(h))$.

So when we backprop through the group head, the feature extractor's gradients are flipped, meaning it **moves away** from representations that let the group head succeed.

In PyTorch, we define a custom autograd **Function**:

```
class GRL(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambda_):
        ctx.lambda_ = lambda_
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        return -ctx.lambda_ * grad_output, None

def grad_reverse(x, lambda_=1.0):
    return GRL.apply(x, lambda_)
```

C. Training Loop

Per batch (x, y_{main}, y_{group}):

1. $h = F(x)$
2. $logits_{main} = M(h) \rightarrow$ main loss: $loss_{main}$
3. $h_{rev} = grad_reverse(h, lambda_grl)$
4. $logits_{group} = G(h_{rev}) \rightarrow$ group loss: $loss_{group}$
5. $loss = loss_{main} + loss_{group}$
6. Backprop: update F, M, G with optimizer.

If debiasing works, after training:

- **Main accuracy** on validation: still good.
 - **Group accuracy** on validation: lower than without GRL (closer to random guess for 3 groups $\approx 33\%$).
-

D. Connecting to the Project

For *Echoes of Equity*:

- **F** will be a CNN over Mel-spectrograms.
 - **M** predicts trustworthiness intent.
 - **G** predicts ethnicity (3 groups).
 - You will tune λ and compare:
 - Baseline CNN without GRL
 - CNN + GRL (various λ)
 - You'll measure:
 - Trustworthiness accuracy, recall per group
 - Fairness gap in recall across White/Black/South Asian
 - Group head accuracy (how predictable ethnicity still is).
-

6. Practice Exercises (5)

Exercise 1 – GRL Concept Check Explain in your own words what GRL does in forward and backward passes, and why this helps debias representations.

Exercise 2 – Architecture Diagram Draw (on paper or in a notebook) the three-part adversarial architecture:

- FeatureExtractor → MainHead
- FeatureExtractor → GRL → GroupHead

Label inputs, outputs, and loss flows.

Exercise 3 – Compare λ Values (Concept) What do you expect to happen if:

- $\lambda = 0.0$
 - $\lambda = 0.5$
 - $\lambda = 2.0$ to main accuracy and group accuracy?
-

Exercise 4 – Baseline Without GRL Modify the training loop to disable GRL (i.e. don't use gradient reversal, or set $\lambda = 0$). Observe:

- Validation main accuracy
 - Validation group accuracy
-

Exercise 5 – With GRL Re-enable GRL with $\lambda = 1.0$ and train again. Compare the validation main and group accuracies with Exercise 4 and write a short paragraph summarizing the differences.

7. Solutions (Model Answers)

Solution 1 – GRL Concept

- Forward: GRL just passes h through unchanged.
 - Backward: GRL multiplies the gradient by $-\lambda$, so the feature extractor is updated in the **opposite direction** for the group loss.
 - Result: The feature extractor learns features that make it **hard** for the group head to predict ethnicity (removing group information) while still learning features useful for the main task.
-

Solution 2 – Architecture Diagram Description

- Input: x (e.g., spectrogram).
 - $x \rightarrow F \rightarrow h$ (shared representation).
 - $h \rightarrow M \rightarrow y_{\text{main_logits}}$ (trustworthy vs neutral).
 - $h \rightarrow \text{GRL} \rightarrow h_{\text{rev}} \rightarrow G \rightarrow y_{\text{group_logits}}$ (ethnicity).
 - Losses:
 - L_{main} backpropagates normally through M and F .
 - L_{group} backpropagates normally through G , but **reversed** through GRL into F .
-

Solution 3 – λ Values (Expected Behavior)

- $\lambda = 0.0$: no adversarial effect; F ignores group loss, so group head can easily predict ethnicity → high group accuracy, possibly more bias.
 - $\lambda = 0.5$: moderate debiasing; group accuracy should drop compared to $\lambda=0$ while main accuracy stays close.
 - $\lambda = 2.0$: very strong debiasing; group accuracy may drop close to random, but if too strong, main accuracy might also deteriorate.
-

Solution 4 – Baseline Without GRL

If you disable GRL, you have:

- $h_{\text{rev}} = h$ (no gradient reversal).
 - F is optimized to be good for both main and group tasks, meaning features explicitly encode ethnicity.
 - Typically you see:
 - High main accuracy (good).
 - High group accuracy (bad for fairness).
-

Solution 5 – With GRL

With $\lambda = 1.0$:

- Group head accuracy on validation should be **lower** (approaching $1/3 \approx 0.33$ for 3 groups).

- Main accuracy might remain similar or slightly lower.
 - Interpretation: features contain **less group information**, which is what we want for debiasing.
-

8. Q&A (10 Common Questions)

1. **Q:** Why don't we just remove ethnicity from the input? **A:** Even if you remove explicit ethnicity labels, models can infer it from other cues (accent, pitch patterns, etc.). GRL helps remove this implicit encoding.
 2. **Q:** Why not simply penalize group accuracy in the loss directly? **A:** That would still train the feature extractor to **help** the group head. GRL flips the gradient so F learns to **hurt** group prediction.
 3. **Q:** Does lower group accuracy always mean better fairness? **A:** It usually means less group information in features, which is helpful, but we still need to check *subgroup performance metrics* (recall, gaps) to be sure.
 4. **Q:** Can adversarial training completely remove bias? **A:** Not necessarily; it can reduce **one type** of bias (encoded demographic info), but data collection, labeling bias, and other factors still matter.
 5. **Q:** Will GRL make my model worse overall? **A:** If λ is set well, main performance usually stays good while fairness improves. If λ is too large, performance may drop too much.
 6. **Q:** Why do we use CrossEntropyLoss for the group head? **A:** Because ethnicity is a multi-class classification problem (3 groups).
 7. **Q:** How many groups can the adversarial head handle? **A:** Any number; you just change the output size of the group head.
 8. **Q:** Can this technique be applied to other tasks? **A:** Yes—any place you want features to be invariant to a specific attribute (e.g., domain, accent, device type).
 9. **Q:** How do judges react to adversarial debiasing? **A:** Very positively, if explained clearly. It shows understanding of both deep learning and fairness.
 10. **Q:** How will we use GRL with CNNs on spectrograms? **A:** Replace the tabular FeatureExtractor with your CNN (from Lesson 4) and attach main and group heads to the CNN's final feature vector.
-

9. Quiz (10 Questions)

1. **What is the main purpose of adversarial debiasing?** → To learn representations that perform well on the main task while being less predictive of sensitive attributes.
2. **What does GRL do during the forward pass?** → Passes inputs through unchanged.
3. **What does GRL do during the backward pass?** → Multiplies gradients by $-\lambda$, reversing their direction.
4. **What are the three main parts of the adversarial network?** → FeatureExtractor, MainHead, GroupHead.

5. **If group accuracy stays very high after adversarial training, what does that suggest?** → The features still encode strong group information; debiasing is weak.
 6. **If λ is set to 0.0, what effect does GRL have?** → No effect; no adversarial pressure.
 7. **What might happen if λ is too large?** → Main task performance may drop significantly.
 8. **Why do we need a group label (y_{group}) at training time?** → The adversarial head needs ground-truth group labels to learn what to predict (and what we want the features to hide).
 9. **Is adversarial debiasing done at training time or inference time?** → Training time; at inference, we only use the feature extractor + main head.
 10. **How do we know if adversarial debiasing is working?** → Group head accuracy goes down while main task accuracy and fairness metrics (like subgroup recall gaps) improve.
-

10. Mini Practice Project – Adversarial Debiasing with GRL (with .zip)

I created a mini project where the student can **see GRL in action** on a synthetic dataset.

Project Goal

- Use synthetic tabular data where **group membership is correlated** with both features and labels.
- Train a small **FeatureExtractor + MainHead + GroupHead** network.
- Use a **Gradient Reversal Layer** so the feature extractor hides group information.
- Explore different λ values to see the trade-off between main accuracy and group accuracy.

What's in the .zip

lesson6_miniproject_adv_debias.zip contains:

- **lesson6_miniproject_adv/**
 - **README.md** – Instructions and experiment ideas.
 - **data/synthetic_adv_data.npz** – Numpy archive with:
 - **X**: (600, 16) feature vectors
 - **y_main**: (600,) main labels (0/1)
 - **y_group**: (600,) group labels (0,1,2 for 3 groups)
 - **src/train_adv_debias.py** – Script including:
 - **GRL** custom autograd function
 - **FeatureExtractor, MainHead, GroupHead**
 - Training loop with **lambda_grl** parameter
 - Prints train/val accuracy for main and group heads each epoch

How to Run

1. Unzip the archive.

2. Install dependencies:

```
pip install torch numpy scikit-learn
```

3. Run:

```
python src/train_adv_debias.py
```

4. Watch printed lines like:

```
Epoch 05 | Train main acc: 0.88, Train group acc: 0.55 | Val main acc: 0.85, Val group acc: 0.38
```

5. Try changing `lambda_gr1` in `train()` to `0.0`, `0.5`, `1.0` and compare:

- How main accuracy changes.
- How group accuracy changes.

This gives an intuitive feel for how strong debiasing affects performance.

11. References

- Adversarial fairness ideas from your project plan (adversarial head to reduce WWIB).
 - Domain-Adversarial Training of Neural Networks (DANN) — classic paper (for more advanced reading).
 - PyTorch custom autograd functions: official docs for extending autograd.
-

12. Additional Information (Mentor Tips)

- Have the student keep a **small results table**: rows = λ values, columns = main accuracy, group accuracy, fairness gap (if they later add subgroup metrics).
- For the final project, they can say something like:

```
"We used an adversarial debiasing architecture with a gradient reversal layer to reduce encoded ethnicity information in the spectrogram representations, which reduced the recall gap between White and South Asian speakers from X% to Y% while maintaining overall accuracy."
```

- That single sentence, backed by proper metrics and plots, will sound extremely strong to ISEF judges.
-

★ Lesson 7 — Adversarial CNN on Spectrograms: From Idea to Full Architecture

1. Lesson Summary

In this lesson, the student will:

- Combine what they learned from:
 - Lesson 3 (Mel-spectrograms),
 - Lesson 4 (CNN classifier),
 - Lesson 6 (adversarial debiasing & GRL),
- Into **one coherent adversarial CNN architecture** for spectrogram-like inputs.
- Understand how to:
 - Use a **CNN feature extractor** on 2D inputs,
 - Attach **main** and **adversarial group** heads,
 - Train with a **Gradient Reversal Layer** to reduce encoded bias.
- Practice on a **synthetic spectrogram dataset** to safely debug the architecture before using the real trustworthiness corpus.

This architecture is a dry run of the real system described in your Echoes of Equity project plan.

2. Key Points

- A **2D CNN** can serve as the shared feature extractor for both main and adversarial tasks.
 - The **main head** predicts trustworthiness (binary).
 - The **group head** predicts ethnicity (multi-class).
 - A **Gradient Reversal Layer (GRL)** is inserted before the group head.
 - During training, we minimize:
 - **L_main** for trustworthiness,
 - **L_group** for ethnicity, but GRL makes the feature extractor **maximize L_group**, erasing group information.
 - We monitor **main accuracy** and **group accuracy** on validation to see the trade-off.
 - Once the student is comfortable with this setup on toy data, they can switch to **real Mel-spectrograms** from the actual dataset.
-

3. Real-World Examples or Stories

- **Domain-adversarial CNNs** are used to make models robust across different accents, microphones, or languages by learning **domain-invariant** features.
 - Your project adapts this idea for **fairness**: instead of "domains," the adversary targets **ethnicity**, with the goal of making features **ethnicity-invariant**.
 - For ISEF, being able to say, "We used an adversarial CNN with gradient reversal to minimize demographic information in the internal representation" is a big intellectual "wow" moment for judges.
-

4. Terminology Explained

- **2D CNN Feature Extractor** – The convolutional layers and pooling layers that turn a 2D spectrogram into a 1D feature vector.
 - **Main Task Head** – A small network (usually linear layers) sitting on top of features to predict the primary label (trustworthiness).
 - **Adversarial Group Head** – A small network predicting group label (ethnicity).
 - **Shared Representation** – The output of the feature extractor that both heads use.
 - **Synthetic Spectrograms** – Fake spectrogram-like images used for debugging and learning; they behave similarly to real ones but are easy to generate.
 - **Joint Training** – Training feature extractor and both heads together in a single optimization loop.
-

5. How It Works (Step-by-Step)

Here we use a **toy setup**: 32×32 “spectrogram-like” images with a binary main label and 3-group demographic label.

A. Data

- **X**: shape (N, 1, 32, 32) – synthetic spectrogram images.
- **y_main**: shape (N,) – binary labels (0/1).
- **y_group**: shape (N,) – group labels (0, 1, 2 for three ethnicities).

In the mini-project data, group-dependent patterns are injected so the network *could* learn ethnicity if we let it.

B. Dataset & DataLoader

```
from torch.utils.data import Dataset, DataLoader
import torch

class AdvCnnDataset(Dataset):
    def __init__(self, X, y_main, y_group):
        self.X = torch.from_numpy(X)
        self.y_main = torch.from_numpy(y_main)
        self.y_group = torch.from_numpy(y_group)

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx], self.y_main[idx], self.y_group[idx]
```

Then split into train/val and wrap with **DataLoader**.

C. CNN Feature Extractor

```
import torch.nn as nn

class CnnFeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 16x16
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 8x8
        )
        self.flatten = nn.Flatten()

    def forward(self, x):
        x = self.conv(x)
        x = self.flatten(x) # shape: (batch, 16*8*8)
        return x
```

This structure mirrors Lesson 4's CNN, but now used as a **shared** extractor.

D. Heads & GRL

Main Head:

```
class MainHead(nn.Module):
    def __init__(self, in_dim=16*8*8):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(in_dim, 32),
            nn.ReLU(),
            nn.Linear(32, 2),
        )

    def forward(self, h):
        return self.fc(h)
```

Group Head:

```
class GroupHead(nn.Module):
    def __init__(self, in_dim=16*8*8, n_groups=3):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(in_dim, 32),
            nn.ReLU(),
            nn.Linear(32, n_groups),
```

```
)

def forward(self, h):
    return self.fc(h)
```

GRL:

```
class GRL(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambda_):
        ctx.lambda_ = lambda_
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        return -ctx.lambda_ * grad_output, None

def grad_reverse(x, lambda_=1.0):
    return GRL.apply(x, lambda_)
```

E. Training Loop (High-Level)

For each batch (xb, yb_main, yb_group):

1. Compute features: `h = feat(xb)`
2. Main path:
 - `logits_main = main_head(h)`
 - `loss_main = CrossEntropy(logits_main, yb_main)`
3. Group path:
 - `h_rev = grad_reverse(h, lambda_grl)`
 - `logits_group = group_head(h_rev)`
 - `loss_group = CrossEntropy(logits_group, yb_group)`
4. Total loss:
 - `loss = loss_main + loss_group`
5. Backprop and optimizer step.
6. Track training & validation accuracy for both heads.

Interpretation:

- If `lambda_grl > 0`, the network is encouraged to **hide group information** while still doing the main task well.

6. Practice Exercises (5)

Exercise 1 – Forward Path Explanation Explain step-by-step what happens when you call:

```
h = feat(xb)
logits_main = main_head(h)
h_rev = grad_reverse(h, lambda_grl)
logits_group = group_head(h_rev)
```

Exercise 2 – Shapes Check Given input batch `xb` with shape `(32, 1, 32, 32)`:

- What are the shapes after:
 - First conv + pool
 - Second conv + pool
 - Flatten
 - Main head output
 - Group head output

Exercise 3 – λ Experiment (Paper Plan) Design a small experiment plan where you will run the model with $\lambda = 0.0, 0.5, 1.0$. For each λ , list the metrics you will record (e.g., main val accuracy, group val accuracy) and what you expect.

Exercise 4 – Turn Off Adversarial Head Conceptually, what changes if you train only the main head (no group head / GRL)? How might this affect fairness compared to the adversarial version?

Exercise 5 – Link to Real Spectrograms Write a short paragraph describing how you would replace the synthetic 32×32 spectrograms with real Mel-spectrograms from the trustworthiness dataset (paths, preprocessing, shapes).

7. Solutions (Model Answers)

Solution 1 – Forward Path Explanation

- `h = feat(xb)` – CNN feature extractor converts each spectrogram into a feature vector.
- `logits_main = main_head(h)` – Main head outputs logits for trustworthy vs neutral.
- `h_rev = grad_reverse(h, lambda_grl)` – Same features in forward pass, but will reverse gradient during backprop.
- `logits_group = group_head(h_rev)` – Adversarial head predicts group (ethnicity) from the gradient-reversed features.

Solution 2 – Shapes

Starting with `(batch=32, channels=1, H=32, W=32)`:

- After Conv1 + ReLU + MaxPool2d(2):
 - Channels: 8, H and W halved: (32, 8, 16, 16)
 - After Conv2 + ReLU + MaxPool2d(2):
 - Channels: 16, H and W halved again: (32, 16, 8, 8)
 - After Flatten:
 - Each sample becomes length $16 \times 8 \times 8 = 1024$: (32, 1024)
 - Main head output:
 - (32, 2) (two classes).
 - Group head output:
 - (32, 3) (three groups).
-

Solution 3 – λ Experiment Plan

For each $\lambda \in \{0.0, 0.5, 1.0\}$:

- Train for 15 epochs with the same random seed.
 - Record:
 - Validation main accuracy (binary).
 - Validation group accuracy (multi-class).
 - Expectations:
 - $\lambda = 0.0 \rightarrow$ high main accuracy, high group accuracy (lots of group info).
 - $\lambda = 0.5 \rightarrow$ similar main accuracy, lower group accuracy.
 - $\lambda = 1.0 \rightarrow$ slightly lower main accuracy, group accuracy closer to random (1/3).
-

Solution 4 – Only Main Head

If you train only the main head:

- There is no pressure to hide group information.
 - The CNN is free to encode whatever helps main accuracy, including group-specific patterns.
 - This may increase fairness issues because internal features could strongly encode ethnicity.
-

Solution 5 – Linking to Real Spectrograms

Example paragraph:

To move from synthetic 32×32 spectrogram images to real Mel-spectrograms, I would first compute Mel-spectrograms from the audio files using librosa, then normalize and resize them (e.g., to 64×64 or 128×128). I would store them as numpy arrays or images, with corresponding labels for trustworthiness

and ethnicity. The CNN feature extractor would be updated to handle the new input size (e.g., $1 \times 64 \times 64$). The training loop would remain the same, except that the DataLoader would now load real spectrogram tensors instead of synthetic ones.

8. Q&A (10 Common Questions)

1. **Q:** Why do we use a CNN instead of a fully connected network here? **A:** Because spectrograms have spatial (time-frequency) structure; CNNs exploit local patterns efficiently.
2. **Q:** How is this different from Lesson 6's adversarial model? **A:** Lesson 6 used a tabular feature extractor; Lesson 7 uses a **CNN** on 2D inputs, which is what we'll use for real spectrograms.
3. **Q:** Do we need to use GRL when evaluating the trained model? **A:** GRL only affects training. At test time, it just passes features through.
4. **Q:** Can we remove the adversarial head at inference time? **A:** Yes. For deployment, you only need the feature extractor + main head.
5. **Q:** How do we know if the adversarial CNN is "better" than a baseline? **A:** Compare main metrics (accuracy, recall) and fairness metrics (e.g., recall per group, fairness gap) against the baseline CNN without GRL.
6. **Q:** Is this architecture heavy for a high schooler's computer? **A:** The synthetic version is tiny. The real version with modest CNN size should still be fine on CPU or Colab.
7. **Q:** Can we use more convolutional layers? **A:** Yes, but start simple to avoid debugging complexity. You can gradually deepen the model later.
8. **Q:** Should we apply data augmentation to spectrograms? **A:** Potentially yes (time/frequency masking), but that's an advanced extension for later lessons.
9. **Q:** Does adversarial training always guarantee fairness? **A:** No guarantee; it reduces one type of encoded bias. You still must evaluate subgroup metrics carefully.
10. **Q:** How do we explain this architecture to non-technical judges? **A:** Something like: "We trained the network to detect trustworthiness while also training a second network that tries to guess the speaker's ethnicity. We then taught the shared representation to hide information from this second network, so the main model focuses on speech patterns rather than demographics."

9. Quiz (10 Questions)

1. **What type of data does the adversarial CNN in this lesson consume?** → 2D spectrogram-like images.
2. **What are the three main modules of the adversarial CNN?** → CNN feature extractor, main head, group head.
3. **What does the main head predict?** → Trustworthiness (binary: trustworthy vs neutral).
4. **What does the group head predict?** → Group/ethnicity (multi-class).

5. **Where is GRL placed in the network?** → Between the feature extractor and the group head.
 6. **What happens to gradients when passing through GRL?** → They are multiplied by $-\lambda$ (reversed and scaled).
 7. **What is the effect of increasing λ (if not too big)?** → Group accuracy usually drops (less group info), while main accuracy can stay reasonably high.
 8. **Why use MaxPool2d in the CNN?** → To reduce spatial resolution and help the network focus on high-level patterns.
 9. **At deployment, which parts of the model are necessary?** → Feature extractor + main head.
 10. **How does this lesson prepare for using the real dataset?** → It validates the full adversarial CNN architecture on toy data, so we can later swap in real spectrograms and trustworthiness/ethnicity labels with confidence.
-

10. Mini Practice Project – Adversarial CNN on Synthetic Spectrograms (.zip)

To make all this concrete, I've created a mini project that trains an **adversarial CNN** on synthetic spectrogram-like images.

Project Goal

- Train a CNN with:
 - Shared convolutional feature extractor,
 - Main classification head,
 - Adversarial group head with GRL.
- Observe how changing `lambda_grl` affects:
 - Main validation accuracy,
 - Group validation accuracy.

What's in the .zip

`lesson7_miniproject_adv_cnn.zip` contains:

- `lesson7_miniproject_adv_cnn/`
 - `README.md` – Instructions and experiment ideas.
 - `data/synthetic_adv_cnn_data.npz` – Numpy archive with:
 - `X`: (N, 1, 32, 32) synthetic spectrogram images
 - `y_main`: (N,) main labels (0/1)
 - `y_group`: (N,) group labels (0,1,2)
 - `src/train_adv_cnn.py` – Script that:

- Defines `CnnFeatureExtractor`, `MainHead`, `GroupHead`, and `GRL`
- Trains the model with `lambda_gr1`
- Prints train/validation main & group accuracy each epoch

How to Run

1. Unzip the archive.
2. Install dependencies:

```
pip install torch numpy scikit-learn
```

3. Run:

```
python src/train_adv_cnn.py
```

4. Then:

- Note main and group accuracy on validation.
- Change `lambda_gr1` in `train()` to `0.0`, `0.5`, `1.0`, re-run, and record results.
- Write a short reflection: which λ seems to best balance main accuracy and reducing group predictability?

11. References

- Echoes of Equity project design sections on **adversarial head and fairness**, and CNN-based models.
 - Domain-Adversarial Training literature (for deeper background): “Domain-Adversarial Training of Neural Networks” (Ganin et al.).
 - PyTorch tutorial on building CNNs for classification.
-

12. Additional Information (Mentor Tips)

- Encourage the student to **save tables** of (λ , main acc, group acc) as this will directly support a figure or table in the final paper.
 - In the actual project, after plugging in real spectrograms, they should:
 - Compare baseline CNN vs adversarial CNN on **subgroup recall**.
 - Show that the adversarial CNN reduces recall gaps while maintaining strong overall performance.
 - For competition judging, this lesson’s content is where the student can confidently talk about:
 - Architecture innovation,
 - Fairness-aware design,
 - Systematic experimentation with hyperparameters (λ).
-

★ Lesson 8 — Experiment Design, Hyperparameter Tuning & Reproducibility

1. Lesson Summary

In this lesson, the student learns how to:

- Design **clean experiments** for the adversarial CNN (baseline vs different λ , learning rates, etc.).
- Tune **hyperparameters** in a structured way instead of guessing.
- Track experiments in a **results table / CSV**, so they can write a convincing **Results & Methods** section for ISEF.
- Think about **reproducibility**: seeds, fixed splits, saving configs and code.

By the end, the student should:

- Have a **small grid of experiments** (e.g., $\lambda = 0, 0.5, 1.0$; LR = $1e-3, 5e-4$, etc.).
 - Be able to answer: "Which configuration best balances performance and fairness, and why?"
-

2. Key Points

- Hyperparameters (learning rate, λ for GRL, batch size, epochs) can **dramatically** impact results.
 - Good experiments vary **one or two hyperparameters at a time** while keeping everything else fixed.
 - Use a **train/validation split** and keep it fixed (same seed) for fair comparisons.
 - Log results in a **structured table** (CSV, Google Sheet): each row = one experiment.
 - Track at least:
 - Main validation accuracy / recall (trustworthy).
 - Group validation accuracy (for adversarial head).
 - Possibly fairness gaps (from Lesson 5).
 - Reproducibility:
 - Fix random seeds.
 - Record code version, dataset version, and hyperparameters.
 - Be able to rerun an experiment and get similar numbers.
 - A well-organized experiment table is something judges and reviewers love.
-

3. Real-World Examples or Stories

- In research labs, ML engineers keep **experiment logs** with dozens or hundreds of runs. Papers often come from noticing a pattern in those logs.

- For fairness research, you might discover that $\lambda = 0.7$ gives almost the same accuracy as $\lambda = 0.0$ but significantly reduces recall gaps. That's publishable insight.
 - In ISEF papers that win top awards, students often show **ablation studies** ("without fairness head vs with fairness head") and **hyperparameter sweeps**, not just a single model.
-

4. Terminology Explained

- **Hyperparameter** – A setting you choose before training (e.g., learning rate, batch size, number of epochs, λ for GRL).
 - **Grid Search** – Trying all combinations from a small set of hyperparameter values (e.g., $LR \in \{1e-3, 5e-4\}$, $\lambda \in \{0, 0.5, 1.0\}$).
 - **Random Seed** – A fixed value given to the random number generator to make experiments repeatable.
 - **Reproducibility** – Ability to rerun code and get (roughly) the same results.
 - **Experiment Config** – A specific combination of hyperparameters and settings used in one run.
 - **Ablation Study** – Systematically removing or changing components (e.g., removing GRL) to see their effect.
 - **Experiment Log / Results Table** – A structured record of each experiment's config and outcome.
-

5. How It Works (Step-by-Step)

Here's a simple plan for hyperparameter experiments on your adversarial CNN:

Step 1 – Choose Hyperparameters to Explore

For example:

- Learning rate: `lr` $\in \{1e-3, 5e-4\}$
- GRL strength: `lambda_grl` $\in \{0.0, 0.5, 1.0\}$

That gives $2 \times 3 = 6$ **experiments** (you can do fewer to start).

Step 2 – Fix the Data Split & Random Seed

- Use `train_test_split(..., random_state=42, stratify=y_group)` and **don't change it**.
- Set `torch.manual_seed(seed)` and `np.random.seed(seed)` for each run.

This ensures differences between runs come from hyperparameters, not random chance.

Step 3 – Define Metrics to Record

For each experiment, record:

- `lr`
- `lambda_grl`
- Validation main accuracy (trustworthiness).
- Validation group accuracy (ethnicity prediction).

Optionally later: fairness metrics per group (from Lesson 5).

Step 4 – Run Experiments in a Loop

You can write a script that:

- Loops over each `(lr, lambda_gr1)` pair.
- Trains the adversarial CNN for a modest number of epochs (e.g., 8–10).
- Evaluates on validation set.
- Writes a row to `experiment_results.csv`.

You'll see patterns like:

- $\lambda = 0 \rightarrow$ main acc high, group acc high (more bias risk).
 - $\lambda = 0.5$ or $1.0 \rightarrow$ group acc lower, main acc slightly lower but still good.
-

Step 5 – Analyze the Table

After you have `experiment_results.csv`, you can:

- Open it in Excel or Google Sheets.
- Sort by main accuracy or group accuracy.
- Choose a configuration that **balances** goals:
 - Main performance high enough,
 - Group accuracy (and later fairness gaps) low enough.

This analysis step becomes a **Result subsection** in the final paper.

6. Practice Exercises (5)

Exercise 1 – Identify Hyperparameters

List at least five hyperparameters you might tune in the Echoes of Equity project and briefly explain what each controls.

Exercise 2 – Design a Grid Search

Suppose you can run exactly **6 experiments**. Propose a grid of `(lr, lambda_gr1)` values that makes sense and explain why.

Exercise 3 – Seed & Split

Explain why we keep the same validation split and random seed across all experiments. What problem happens if we don't?

Exercise 4 – Results Table Sketch

Draw a small table with columns and 3 example rows for:

- `lr`, `lambda_grl`, `val_main_acc`, `val_group_acc`.

Fill in fake numbers that illustrate the kind of pattern you expect (e.g., λ higher $\rightarrow$ group acc lower).

Exercise 5 – Reproducibility Checklist

Write a checklist of at least 6 items that you should track to make your experiments reproducible (for yourself and future readers).

7. Solutions (Model Answers)

Solution 1 – Example Hyperparameters

- Learning rate (`lr`) – How big each gradient step is.
 - Batch size – How many samples per gradient update.
 - Number of epochs – How many passes over the training set.
 - `lambda_grl` – Strength of adversarial debiasing.
 - CNN architecture choices – number of filters, kernel size, number of convolutional layers.
 - Weight decay (L2 regularization) – How strongly we penalize large weights.
-

Solution 2 – Grid Search Example

With 6 experiments, one simple grid:

- `lr` $\in \{1e-3, 5e-4\}$
- `lambda_grl` $\in \{0.0, 0.5, 1.0\}$

This explores both a standard learning rate and a slightly smaller one, and three levels of debiasing (none, medium, strong). It's small but covers key trade-offs.

Solution 3 – Seed & Split

We keep the seed and split fixed so:

- All models see **exactly the same training and validation data**.
- Differences in metrics are due mostly to hyperparameters, not random differences in which samples went to train vs validation.

If we don't fix them:

- Some models might "get lucky" with an easier validation split, leading to **unfair comparisons**.
-

Solution 4 – Results Table Sketch

Example:

lr	lambda_gri	val_main_acc	val_group_acc
1e-3	0.0	0.88	0.80
1e-3	0.5	0.86	0.55
5e-4	1.0	0.84	0.40

This shows a reasonable pattern: higher λ ($0 \rightarrow 1.0$) tends to lower group accuracy (less group info), with a small trade-off in main accuracy.

Solution 5 – Reproducibility Checklist

Example checklist:

1. Dataset version and preprocessing steps.
 2. Train/validation split (including `random_state`).
 3. Random seeds for numpy and PyTorch.
 4. Model architecture (layers, sizes, activation functions).
 5. Hyperparameters (lr, batch size, epochs, λ , weight decay, etc.).
 6. Code version (Git commit hash, file version).
 7. Exact command used to run the experiment.
 8. Metrics recorded and where they're stored (CSV path).
-

8. Q&A (10 Common Questions)

1. **Q:** How many experiments do I need for ISEF? **A:** Quality > quantity. Even 6–10 well-designed experiments with clear analysis is much better than 50 random runs with no structure.
2. **Q:** What if results change slightly even with fixed seed? **A:** Small variations are normal (e.g., due to GPU nondeterminism). You just want them to be **similar**, not identical.
3. **Q:** Should I always pick the model with the highest main accuracy? **A:** Not necessarily. In a fairness project, you may prefer a slightly lower accuracy with much better fairness metrics.
4. **Q:** Is grid search the only way? **A:** No. There's random search, Bayesian optimization, etc. But grid search is simple and enough for this project.
5. **Q:** How do I show these experiments in my paper/poster? **A:** Use a table of configs vs metrics, and a short discussion of trends (e.g., "As λ increased, group accuracy decreased while main accuracy remained above 84%").
6. **Q:** What if one experiment performs much worse? **A:** That's still useful! You can explain why that configuration is not good and what it taught you.
7. **Q:** Do I need a separate test set beyond validation? **A:** Ideally yes for a final unbiased performance estimate, but for ISEF, a train/val split plus careful methodology is often acceptable.
8. **Q:** How long should each experiment run? **A:** Long enough to converge reasonably (e.g., 8–20 epochs), but not so long that you can't iterate. Use a smaller model or dataset if needed.

9. **Q:** Can I reuse hyperparameters from other related papers? **A:** Yes, you can start from them and then do a small local search around those values.
10. **Q:** How do I keep my experiment logs organized over months? **A:** Use a consistent naming convention (experiment IDs), store CSVs in a dedicated folder, and consider a simple spreadsheet summarizing everything.
-

9. Quiz (10 Questions)

1. **What is a hyperparameter?** → A configuration value set before training, like learning rate or λ , that the model does not learn automatically.
 2. **Why do we use a validation set?** → To evaluate and compare different hyperparameters without touching the eventual test set.
 3. **What is grid search?** → Systematically trying all combinations of a small set of hyperparameter values.
 4. **Why is it important to fix random seeds?** → To make results repeatable and ensure differences come from hyperparameters, not randomness.
 5. **What is an ablation study?** → Systematically removing or changing parts of the model (e.g., GRL) to see their effect on performance.
 6. **Which of these is *not* a hyperparameter?** A) Learning rate B) Model weights C) Number of epochs D) λ for GRL → Correct: **B) Model weights** (they are learned, not preset).
 7. **Where should you store experiment results?** → In a structured format like a CSV / spreadsheet with one row per experiment.
 8. **What is one sign of poor experiment design?** → Changing many things at once, so you can't tell what caused differences in results.
 9. **Why might you choose $\lambda = 0.5$ instead of $\lambda = 0.0$?** → To introduce some debiasing while keeping main accuracy reasonably high.
 10. **What does "reproducible experiment" mean?** → Another person (or future you) can rerun it with the same code, data, and hyperparameters and get similar results.
-

10. Mini Practice Project – Experiment Tracking & Hyperparameter Tuning (.zip)

To make this practical, I created a mini project where the student actually **runs multiple experiments** and logs results.

Project Goal

- Run several adversarial CNN experiments with different:
 - Learning rates (`lr`)
 - GRL strengths (`lambda_grl`)

- Track:
 - Validation main accuracy
 - Validation group accuracy
- Save everything to `experiment_results.csv` and interpret the trade-offs.

What's in the .zip

`lesson8_miniproject_experiments.zip` contains:

- `lesson8_miniproject_experiments/`
 - `README.md` – Instructions and reflection prompts.
 - `data/synthetic_experiment_data.npz` – Synthetic spectrogram-like dataset:
 - `X`: (N, 1, 32, 32)
 - `y_main`: binary labels
 - `y_group`: 3-group labels
 - `src/run_experiments.py` – Script that:
 - Defines an adversarial CNN (feature extractor + main + group heads with GRL).
 - Loops over several (lr, lambda_grl) configs.
 - Trains each for a few epochs.
 - Logs `lr`, `lambda_grl`, `val_main_acc`, `val_group_acc` to `data/experiment_results.csv`.

How to Run

1. Unzip the archive.
2. Install dependencies:

```
pip install torch numpy scikit-learn
```

3. Run:

```
python src/run_experiments.py
```

4. Open `data/experiment_results.csv` in Excel / Sheets.
5. Answer questions like:
 - Which configuration has the **best main accuracy**?
 - Which configuration leads to the **lowest group accuracy**?
 - Which setup offers the **best balance**?

These answers can later be adapted into the “Hyperparameter Tuning” or “Model Selection” subsection of the final paper.

11. References

- General ML experiment design: any section on evaluation & tuning in an ML textbook (e.g., *Hands-On Machine Learning with Scikit-Learn & TensorFlow*).
 - Reproducibility tips from ML blogs / talks (e.g., conference tutorials on reproducible ML).
 - Fairness papers often include hyperparameter/ablation tables—great to skim for formatting ideas.
-

12. Additional Information (Mentor Tips)

- Encourage the student to **name experiments** (e.g., `cnn_lambda1_1r1e-3`) and maybe keep a simple Google Sheet summarizing the best ones.
 - For the final ISEF report, I would recommend:
 - A table of 5–10 experiments with short captions.
 - A paragraph explaining the trend: “As λ increased, group accuracy decreased from X to Y, while main accuracy remained above Z.”
 - This lesson also secretly trains them in **research discipline**, which will be incredibly valuable for future college research and internships.
-

★ Lesson 9 — Explainability: What Is the Model “Listening To”?

1. Lesson Summary

In this lesson, the student will:

- Learn why **explainability** is critical for a fairness project like *Echoes of Equity*.
- Understand how to use **Grad-CAM** (Gradient-weighted Class Activation Mapping) to highlight **which parts of a spectrogram** influenced a prediction.
- Connect explainability to **fairness**:
 - Are we focusing on “trustworthiness prosody” or demographic cues?
- Practice on a **synthetic spectrogram dataset**:
 - Train a small CNN,
 - Generate Grad-CAM heatmaps,
 - Interpret them qualitatively.

These skills will later be applied to the real Mel-spectrograms for the ISEF paper/poster.

2. Key Points

- Explainability helps answer: **“Why did the model call this sample trustworthy?”**
 - **Grad-CAM** uses gradients of the target class with respect to feature maps in a convolutional layer to produce a **heatmap of importance**.
 - For spectrograms, Grad-CAM shows **time–frequency regions** the model relies on.
 - In a fairness project, we care if heatmaps differ **systematically** between groups in suspicious ways (e.g., focusing mainly on portions correlated with accent or background noise).
 - Combining:
 - **Quantitative metrics** (accuracy, fairness gaps), and
 - **Qualitative visualizations** (Grad-CAM) makes the scientific argument much stronger.
 - Judges love clear visuals: “Before debiasing vs after debiasing” heatmaps for the same utterance.
-

3. Real-World Examples or Stories

- In medical imaging, Grad-CAM heatmaps show radiologists whether a model is focusing on the actual lesion or irrelevant artifacts.
 - In speech emotion recognition, Grad-CAM on spectrograms can reveal which **pitch or energy patterns** matter most for detecting anger, happiness, etc.
 - For *Echoes of Equity*, you can show heatmaps for:
 - Baseline CNN (no debiasing) vs
 - Adversarially debiased CNN, on the **same audio sample**, and discuss if the attention shifts toward more “universal” trust cues.
-

4. Terminology Explained

- **Explainability / Interpretability** – Methods to understand how a model makes its decisions.
 - **Grad-CAM** – A technique that uses gradients backpropagated from a target class to create a **class-specific saliency map** over the feature maps of a CNN.
 - **Activation Maps / Feature Maps** – Outputs of convolution layers that capture local patterns in the input.
 - **Heatmap** – A color-coded image where bright/high values represent important regions and dark/low values less important regions.
 - **Saliency** – How much each part of the input contributes to the output.
 - **Overlay** – Plotting the heatmap on top of the original spectrogram to make interpretation easier.
-

5. How It Works (Step-by-Step)

We'll use a tiny CNN on 32×32 spectrogram-like images (synthetic) to demonstrate Grad-CAM.

A. Model Structure

```
class SimpleCnn(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2), # 8x8
        )
        self.flatten = nn.Flatten()
        self.fc = nn.Sequential(
            nn.Linear(16 * 8 * 8, 32),
            nn.ReLU(),
            nn.Linear(32, 2), # 2 classes
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.flatten(x)
        x = self.fc(x)
        return x
```

We will compute Grad-CAM with respect to the **last conv layer** (`self.conv[-2]`, the second `Conv2d`).

B. Intuition of Grad-CAM

1. Pick an input `x` and a **target class** (e.g., predicted class).
2. Run forward pass → get feature maps `A` from chosen conv layer and logits `y`.
3. Compute gradients of `y[target_class]` w.r.t. `A`.
4. Average gradients over spatial dimensions → “importance weights” for each channel.
5. Compute weighted sum of feature maps using these weights → class activation map.
6. Apply ReLU, normalize, and upsample to input resolution.
7. Overlay on original spectrogram.

C. Implementation Sketch

Hooks to capture activations and gradients:

```
conv_layer = model.conv[-2]
activations = []
gradients = []
```

```
def forward_hook(module, inp, out):
    activations.append(out.detach())

def backward_hook(module, grad_in, grad_out):
    gradients.append(grad_out[0].detach())

handle_f = conv_layer.register_forward_hook(forward_hook)
handle_b = conv_layer.register_backward_hook(backward_hook)
```

Compute Grad-CAM:

```
x = x.to(device)
x.requires_grad_(True)

logits = model(x)
loss = logits[0, target_class]
model.zero_grad()
loss.backward()

act = activations[0]      # (1, C, Hc, Wc)
grad = gradients[0]      # (1, C, Hc, Wc)

weights = grad.mean(dim=(2, 3), keepdim=True) # (1, C, 1, 1)
cam = (weights * act).sum(dim=1)              # (1, Hc, Wc)
cam = torch.relu(cam[0])

cam -= cam.min()
if cam.max() > 0:
    cam /= cam.max()
```

Upsample and overlay:

```
cam_upsampled = torch.nn.functional.interpolate(
    cam.unsqueeze(0).unsqueeze(0),
    size=x.shape[2:], mode="bilinear", align_corners=False
)[0, 0]

heatmap = cam_upsampled.detach().cpu().numpy()
```

Then plot original + overlay:

```
axes[0].imshow(x_np[0, 0], aspect="auto", origin="lower")
axes[1].imshow(x_np[0, 0], cmap="gray", origin="lower", aspect="auto")
axes[1].imshow(heatmap, cmap="jet", alpha=0.5, origin="lower", aspect="auto")
```

D. Connecting to Echoes of Equity

On the **real project**:

- Compute Grad-CAM for:
 - Baseline CNN (no adversarial head), and
 - Debaised CNN (with GRL), on the **same audio**.
 - Ask:
 - Are we focusing on similar regions?
 - Does the debaised network move away from obviously group-specific artifacts?
 - Show these pairs of heatmaps on the ISEF board under “**Model Interpretation & Fairness Evidence**”.
-

6. Practice Exercises (5)

Exercise 1 – Explainability Motivation In your own words, explain why visual explainability is especially important in a fairness-focused project like Echoes of Equity.

Exercise 2 – Grad-CAM Intuition Without math, describe what Grad-CAM tells you about a model’s prediction on a spectrogram.

Exercise 3 – Where to Hook Which layer of the CNN is usually a good choice for Grad-CAM, and why not the very first or very last layer?

Exercise 4 – Misclassification Analysis Suppose the model misclassifies a sample (predicts trustworthy but it’s actually neutral). How could a Grad-CAM heatmap help you analyze the error?

Exercise 5 – Fairness Hypothesis Write a short hypothesis: how you would expect Grad-CAM heatmaps to **change** before vs after adversarial debiasing for the same speaker.

7. Solutions (Model Answers)

Solution 1 – Explainability Motivation

Explainability lets us see *how* the model is using the audio. In a fairness project, we want confidence that the model focuses on **trust-related prosody** instead of unwanted demographic cues. Grad-CAM visualizations help judges and users trust that your fairness methods actually changed what the model listens to.

Solution 2 – Grad-CAM Intuition

Grad-CAM highlights which parts of the spectrogram had the **largest influence** on a specific prediction. Bright areas in the heatmap are where the model was “paying attention” when deciding “trustworthy” vs “neutral.”

Solution 3 – Where to Hook

A late convolutional layer (e.g., the last conv block) is usually best:

- Early layers detect very low-level patterns (like edges/noise), which are too fine-grained.
 - The very last fully connected layer loses spatial structure.
 - Late conv layers still have spatial layout but represent higher-level patterns.
-

Solution 4 – Misclassification Analysis

Grad-CAM can show that the model focused on an irrelevant burst of noise or a short segment where the speaker raised their voice, instead of the overall steady tone. This can suggest ideas for data cleaning, augmentation, or architecture changes.

Solution 5 – Fairness Hypothesis

Example hypothesis:

Before adversarial debiasing, Grad-CAM heatmaps might focus on regions that correlate with certain accents or speaking styles typical of one group. After debiasing, heatmaps should become more similar across groups, emphasizing core trust cues (steady pitch, moderate intensity) rather than group-specific quirks.

8. Q&A (10 Common Questions)

1. **Q:** Is Grad-CAM the only explainability method? **A:** No. There are others like vanilla saliency, Integrated Gradients, LIME, SHAP, etc. Grad-CAM is convenient for CNNs on images/spectrograms.
2. **Q:** Does Grad-CAM change how the model behaves? **A:** No, it just analyzes the model after training; it doesn't alter weights.
3. **Q:** Can Grad-CAM prove the model is fair? **A:** Not by itself. It provides qualitative evidence that supports quantitative fairness metrics.
4. **Q:** What if the heatmaps look noisy? **A:** That may mean the model is not very confident or the architecture is too small. Training longer or smoothing can help.
5. **Q:** Can we use Grad-CAM on the adversarial head? **A:** Yes, in theory you could visualize what features help the group classifier and compare them to the main task's focus.
6. **Q:** Do we need GPU for Grad-CAM? **A:** No, CPU is fine for small models, just slower.
7. **Q:** How many example heatmaps should we show in the paper/poster? **A:** A handful of **carefully chosen cases** (e.g., one per group, before/after debiasing) is better than dozens of similar images.
8. **Q:** Can Grad-CAM be wrong? **A:** It's an approximation, so don't over-interpret single pixels. Look for broader patterns and validate with domain knowledge.

9. **Q:** How do we choose which samples to visualize? **A:** Good choices: typical correct predictions, interesting edge cases, and misclassified examples.
10. **Q:** Does Grad-CAM work for non-CNN models? **A:** It's primarily designed for CNNs. Other methods are more suitable for transformers or tabular models.
-

9. Quiz (10 Questions)

1. **What is the main purpose of Grad-CAM?** → To highlight which regions of an input (image/spectrogram) are most important for a specific class prediction.
 2. **Which part of the model do we typically apply Grad-CAM to?** → A late convolutional layer.
 3. **What information does Grad-CAM use to compute the heatmap?** → Gradients of the target class score with respect to the feature maps.
 4. **Why is Grad-CAM especially suitable for spectrograms?** → Spectrograms are 2D like images, and Grad-CAM produces 2D heatmaps aligned with them.
 5. **Does Grad-CAM require retraining the model?** → No, it only requires access to gradients during evaluation.
 6. **How can Grad-CAM support fairness analysis?** → By showing whether the model is focusing on similar prosodic regions across different demographic groups.
 7. **What does a brighter region in a Grad-CAM heatmap indicate?** → A region that strongly contributed to the model's decision for that class.
 8. **What is the role of ReLU in Grad-CAM?** → It keeps only positive influences for the target class, making the heatmap easier to interpret.
 9. **Why do we normalize the heatmap to [0, 1]?** → To map it cleanly into color scales and compare across samples.
 10. **What is a good way to present Grad-CAM results in your ISEF poster?** → Side-by-side panels: original spectrogram vs Grad-CAM overlay, with clear labels and a short caption explaining what the model is focusing on.
-

10. Mini Practice Project – Grad-CAM on Synthetic Spectrograms (.zip)

To make this hands-on, I prepared a mini project where the student:

- Trains a small CNN on synthetic spectrogram-like images, and
- Uses Grad-CAM to visualize what the model is "listening to."

Project Goal

- Learn how to:
 - Attach hooks to CNN layers,

- Compute Grad-CAM heatmaps,
- Save PNG visualizations,
- Interpret them.

What's in the .zip

lesson9_miniproject_explain.zip contains:

- **lesson9_miniproject_explain/**
 - **README.md** – Step-by-step instructions.
 - **data/synthetic_explain_data.npz** – Synthetic spectrogram-like dataset:
 - **X**: (N, 1, 32, 32)
 - **y_main**: (N,) binary labels (class 0 vs 1)
 - **y_group**: (N,) group labels (not used here but included for realism)
 - **src/gradcam_demo.py** – Script that:
 - Trains **SimpleCnn** for a few epochs.
 - Computes Grad-CAM for a few validation samples.
 - Saves side-by-side original + Grad-CAM images in **outputs/**.

How to Run

1. Unzip the archive.
2. Install dependencies:

```
pip install torch numpy scikit-learn matplotlib
```

3. Run:

```
python src/gradcam_demo.py
```

4. Open the **outputs/** folder and view images like **example_0_gradcam.png**.

Prompt the student:

- Describe where the red/yellow regions are.
- Are they aligned with the obvious “class pattern” in the synthetic spectrogram?
- How might this look different for real trustworthiness spectrograms?

11. References

- Echoes of Equity project outline: fairness-aware CNN and visualization deliverables.

- Original Grad-CAM paper (for advanced reading): “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.”
- Example tutorials: “Grad-CAM with PyTorch” (various blog posts and GitHub examples).

12. Additional Information (Mentor Tips)

- For the real project, I’d recommend:
 - Choose **3–6 representative samples** (across groups) and show heatmaps for:
 - Random Forest (no visualization, just for contrast),
 - Baseline CNN,
 - Adversarially debiased CNN.
- For the paper/poster, include a small “**Methods: Explainability**” section explaining Grad-CAM in 3–4 sentences plus a diagram.
- Encourage the student to keep a **notebook of interesting Grad-CAM cases** (especially misclassifications) and reflect on whether they align with their fairness story.

★ Lesson 10 — Final Evaluation, Storytelling & ISEF-Ready Deliverables

1. Lesson Summary

In this final lesson, the student will:

- Learn how to **run a final evaluation** of baseline vs debiased models.
- Compute **overall performance** and **fairness metrics** (per-group recall, fairness gap).
- Turn numbers into a **clear scientific story** suitable for:
 - ISEF paper,
 - Poster,
 - Oral presentation.
- Organize all code, data, and figures so the project is **reproducible and polished**.
- Practice generating a **short written report** summarizing results and conclusions.

By the end of Lesson 10, the student should be ready to:

“Ship” Echoes of Equity as a complete ISEF project: code, experiments, fairness analysis, and explanation.

2. Key Points

- Final evaluation compares at least **two models**:
 - Baseline CNN (no fairness mechanisms),
 - Debiased CNN (adversarial GRL, tuned hyperparameters).
 - Core metrics to report:
 - **Overall accuracy / F1 / recall**,
 - **Per-group recall** (for trustworthiness),
 - **Fairness gap**: max difference between group recalls.
 - The **story**: Did the debiased model:
 - Maintain similar (or acceptable) accuracy?
 - Reduce fairness gap across groups?
 - Organize all:
 - Scripts (data prep, training, evaluation, Grad-CAM),
 - Results tables & plots,
 - Notes and interpretations.
 - For ISEF, clarity and honesty matter:
 - Show strengths **and** limitations,
 - Suggest future improvements.
-

3. Real-World Examples or Stories

- Published fairness papers typically include **baseline vs debiased** comparisons with clear metrics and sometimes trade-offs (e.g., small drop in accuracy for big fairness gain).
 - Top ISEF projects often show:
 - A baseline model,
 - A modified/improved model,
 - Carefully discussed results—not just “my accuracy is 0.95”, but **what it means** and for **whom**.
 - Your project’s narrative:
 - “We started with a naive trustworthiness detector that worked well overall but was biased toward White speakers. We then designed an adversarial CNN to reduce this effect, and showed that fairness metrics improved while accuracy stayed competitive.”
-

4. Terminology Explained

- **Baseline Model** – The simplest reasonable model used as a reference (e.g., CNN without adversarial head).
- **Debiased Model** – The model incorporating fairness techniques (e.g., adversarial CNN with GRL, tuned λ).

- **Per-Group Recall** – For each demographic group, the proportion of actual “trustworthy” samples that the model correctly identifies as trustworthy.
- **Fairness Gap** – The difference between the highest and lowest per-group recall; smaller is better for fairness.
- **Final Test Set** – Data held out until the end, used only for final evaluation.
- **Scientific Conclusion** – A statement about what the results show, with caveats and limitations.

5. How It Works (Step-by-Step)

A. Final Evaluation Plan

1. Choose your **final models**:
 - Baseline CNN: best configuration without GRL.
 - Debiased CNN: chosen after Lesson 8 experiment tuning (balanced λ).
2. Evaluate both on the **same test set**:
 - Overall accuracy / F1.
 - Per-group recall for trustworthiness.
 - Fairness gap.
3. Summarize in a **comparison table** and short narrative.

B. Computing Metrics (Concept)

For each model:

- **Accuracy**:
$$[\text{Accuracy}] = \frac{\text{\# correct predictions}}{\text{\# all samples}}$$
- **Per-group recall (group G, for trustworthy=1)**:
$$[\text{Recall}]_G = \frac{\text{\# of true positives in group G}}{\text{\# of actual positives in group G}}$$
- **Fairness gap**:
$$[\text{Gap}] = \max_G([\text{Recall}]_G) - \min_G([\text{Recall}]_G)$$

Smaller gap means more equal treatment across groups.

C. Example Comparison Table

Model	Accuracy	Recall(White)	Recall(Black)	Recall(SouthAsian)	Fairness Gap
Baseline	0.88	0.92	0.80	0.76	0.16
Debiased	0.86	0.88	0.84	0.83	0.05

Story: slight drop in accuracy, **much smaller fairness gap** → success.

D. Turning Numbers into a Story

A good paragraph might look like:

“Our baseline CNN achieved 0.88 accuracy, but recall varied widely across groups (White: 0.92, Black: 0.80, South Asian: 0.76; fairness gap 0.16). After adding adversarial debiasing with a gradient reversal layer ($\lambda = 0.7$), our debiased CNN achieved 0.86 accuracy while substantially reducing the recall gap to 0.05 (White: 0.88, Black: 0.84, South Asian: 0.83). This suggests the debiased model treats speakers from different backgrounds more equally while maintaining strong performance.”

E. Checklist for ISEF-Ready Deliverables

- ☒ Working code (data prep, training, evaluation, Grad-CAM).
- ☒ Saved metrics and experiment tables (CSV).
- ☒ Clear baseline vs debiased comparison.
- ☒ Plots: fairness gap bar chart, overall metrics.
- ☒ 1–2 Grad-CAM visual examples (before vs after).
- ☒ Written report sections: Abstract, Methods, Results, Discussion, Limitations, Future Work.

6. Practice Exercises (5)

Exercise 1 – Metric Definitions

Write the formulas for accuracy, per-group recall, and fairness gap in your own words, and explain what each tells you.

Exercise 2 – Fake Comparison Table

Create a small table (like the one above) with made-up numbers where the debiased model slightly lowers accuracy but significantly improves fairness gap.

Exercise 3 – Draft a Conclusion Paragraph

Based on your fake table, write 4–6 sentences describing what happened, focusing on both accuracy and fairness.

Exercise 4 – Limitations & Future Work

List at least 3 limitations of your (imagined) project and 3 ideas for future work.

Exercise 5 – Poster Outline

Outline 5 sections you would include on the ISEF poster for Echoes of Equity and 1–2 bullet points under each.

7. Solutions (Model Answers)

Solution 1 – Metric Definitions (Plain English)

- Accuracy: “Out of all predictions, how many did we get right?”
- Per-group recall: “Within each group, among the people who truly were trustworthy, what fraction did we correctly label as trustworthy?”
- Fairness gap: “How far apart are the best and worst per-group recalls? If the gap is big, one group is treated much better than another.”

Solution 2 – Example Table

Model	Accuracy	Recall(White)	Recall(Black)	Recall(SouthAsian)	Fairness Gap
Baseline	0.90	0.94	0.82	0.78	0.16
Debiased	0.88	0.90	0.86	0.84	0.06

Solution 3 – Sample Conclusion Paragraph

“We compared a baseline CNN to an adversarially debiased CNN. The baseline model achieved 0.90 accuracy, but its recall fairness gap was 0.16, meaning some groups were recognized as trustworthy far more often than others. After adding our fairness mechanism, the debiased model’s accuracy dropped slightly to 0.88, but the recall gap decreased to 0.06. This suggests that our approach substantially reduced disparities in how different groups are treated. The debiased model still maintains strong overall performance while being noticeably fairer.”

Solution 4 – Limitations & Future Work

Limitations:

1. Dataset size may be limited, especially for some demographic groups.
2. Labels for “trustworthy” vs “neutral” are subjective and might encode human bias.
3. Only one fairness technique (adversarial debiasing) was tested.

Future Work:

1. Collect a larger and more diverse dataset, especially for underrepresented groups.
2. Explore additional fairness methods (e.g., re-weighting, post-processing).
3. Involve human subjects to evaluate whether the model’s decisions feel fair in practice.

Solution 5 – Poster Outline

1. Introduction & Motivation

- Why voice-based trustworthiness detection is interesting and risky.
- What fairness means in this context.

2. Data & Problem Setup

- Dataset description, groups, labels.
- Example spectrograms.

3. **Methods: Models & Fairness Techniques**

- Baseline CNN architecture.
- Adversarial debiasing with GRL.

4. **Results & Analysis**

- Tables of accuracy and per-group recall.
- Fairness gap plots, Grad-CAM visualizations.

5. **Conclusion & Future Work**

- Summary of improvements.
- Limitations and next steps.

8. Q&A (10 Common Questions)

1. **Q:** What if the debiased model's accuracy drops a lot? **A:** Then the trade-off may be too severe. You might need to adjust λ or try alternative methods. Be honest about this in your report.
2. **Q:** Do we always need a baseline? **A:** Yes, for science you need something to compare against. Otherwise you can't claim improvement.
3. **Q:** Is fairness gap the only fairness metric? **A:** No, but it's a simple, intuitive one. You can also look at balanced accuracy, equalized odds, etc.
4. **Q:** Should we tune hyperparameters on the test set? **A:** No. Use validation data for tuning; keep test set only for final evaluation.
5. **Q:** How many decimal places should we report? **A:** Usually 2 or 3 decimals is enough. More doesn't add real meaning.
6. **Q:** Can we include negative results? **A:** Yes. "We tried X and it didn't help" is still valuable information.
7. **Q:** How do we avoid cherry-picking? **A:** Decide in advance which metrics and comparisons you will report and show all of them, not only the best-looking ones.
8. **Q:** What if different runs give slightly different results? **A:** That's normal. You can mention variability and possibly average results over a few runs.
9. **Q:** How long should the final written report be? **A:** Follow ISEF or competition guidelines, but typically 5–15 pages including figures is common for high school research.
10. **Q:** What's the most important thing judges look for? **A:** Clear reasoning, honest analysis, understanding of limitations, and that the student really understands their own work.

9. Quiz (10 Questions)

1. **What is the main purpose of a baseline model?** → To provide a reference for comparison so we can tell if our new method actually improved anything.

2. **Why is per-group recall important in fairness analysis?** → It shows how well the model serves each demographic group, not just the population overall.
 3. **How do we define fairness gap in this context?** → The difference between the highest and lowest per-group recall.
 4. **If the debiased model's fairness gap is much smaller but accuracy is slightly lower, is that always bad?** → No, it may be a good trade-off if fairness is a priority and accuracy remains acceptable.
 5. **Why keep the same test set for both baseline and debiased models?** → To ensure a fair comparison —both models are judged on identical data.
 6. **What does “reproducible” mean for your final results?** → Others (or future you) can run the same code and get similar metrics.
 7. **Why should we include limitations in the report?** → It shows honesty, scientific maturity, and helps future work build on your project.
 8. **What is one risk of tuning hyperparameters on the test set?** → Overfitting to the test set, giving overly optimistic results.
 9. **How can Grad-CAM support your final conclusions?** → By visually showing that the model focuses on similar trust-related regions across groups, consistent with improved fairness metrics.
 10. **What is the final goal of Lesson 10?** → To assemble a complete, coherent, ISEF-ready project: metrics, code, visualizations, and a clear scientific narrative.
-

10. Mini Practice Project – Final Evaluation & Report Generator (.zip)

To give the student a concrete template for their **final evaluation and reporting**, I prepared a small mini project.

Project Goal

- Practice:
 - Computing overall accuracy and per-group recall for **baseline vs debiased** models.
 - Computing a simple **fairness gap**.
 - Auto-generating a short **markdown report** summarizing results and interpretation prompts.

What's in the .zip

lesson10_miniproject_final.zip contains:

- **lesson10_miniproject_final/**
 - **README.md** – Instructions.
 - **data/final_eval_data.csv** – Synthetic evaluation dataset with columns:
 - **sample_id**
 - **group** (White, Black, SouthAsian)

- `y_true` (0/1 true trustworthiness)
- `y_pred_baseline` (predictions from a simulated baseline)
- `y_pred_debiased` (predictions from a simulated debiased model)
- `src/final_report.py` – Script that:
 - Loads the CSV.
 - Computes overall accuracy, per-group recall, and fairness gap for both models.
 - Prints a terminal summary.
 - Writes a Markdown file `final_report.md` with tables and a section to write conclusions.

How to Run

1. Unzip the archive.
2. Install dependency:

```
pip install pandas
```

3. Run:

```
python src/final_report.py
```

4. Check:

- The printed comparison in your terminal.
- The generated `final_report.md`, and edit the **Interpretation** section as practice.

This is a **small-scale rehearsal** of what the student will do with their **real** Echoes of Equity test set and models.

11. References

- Your Echoes of Equity project plan (sections on evaluation, fairness metrics, and ISEF positioning).
- Any standard ML text or online guide on model evaluation and fairness metrics.

12. Additional Information (Mentor Tips)

- Help the student explicitly **separate**:
 - Engineering work (coding and training),
 - Scientific work (defining hypotheses, designing experiments, analyzing results).
- For ISEF preparation, I'd suggest:
 - Doing a mock **10-minute presentation** with slides or poster.

- Practicing answers to likely judge questions:
 - “What is your baseline?”
 - “How do you measure fairness?”
 - “What are the limitations of your dataset?”
 - “What would you try next if you had more time?”
 - Make sure all final code and data are backed up (e.g., GitHub + Google Drive) and that the student can re-run at least one full pipeline end-to-end before the fair.
-